

很大，当系统在传输该文件时其他请求都被阻塞了。

一种显而易见的改进方法（所有的 Web 服务器都采用了这种方法）是在内存中维护一个缓存，其中保存着 n 个最近使用过的文件或者数千兆量的内容。服务器在从磁盘读取文件之前，首先检查缓存。如果缓存中有该文件，则直接从内存中取出文件，从而消除了访问磁盘的时间。虽然真正有效的缓存需要大量的内存，以及一定的额外处理时间来完成检查缓存和管理其内容，但是，节省下来的时间几乎总是抵得上这些开销和费用。

为了解决一次只能服务一个请求的问题，一种策略是将服务器设计成多线程模式（multithreaded）。在其中一种设计方案中，服务器由一个前端模块（front-end module）和 k 个处理模块（processing module）组成，如图 7-21 所示。前端模块接受所有入境请求； $k+1$ 个线程全部属于同一个进程，这样所有处理模块都可以访问当前进程地址空间中的缓存。当一个请求到达时，前端模块接受它，并为其创建一条描述该请求的简短记录，然后将该记录递交给其中一个处理模块。

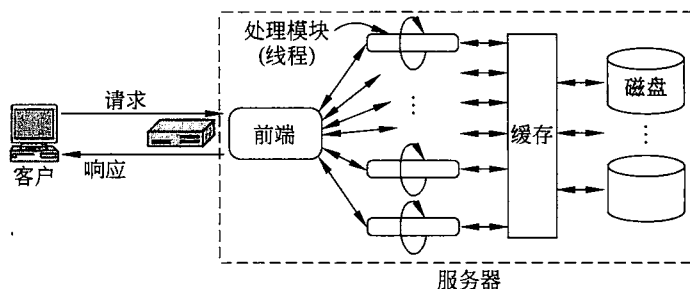


图 7-21 具有一个前端和若干个处理模块的多线程 Web 服务器

处理模块首先检查缓存，查看其中是否有所需的文件。如果缓存中有该文件，则处理模块修改记录，在记录中增加一个指向该文件的指针；如果缓存中没有该文件，则处理模块执行一次磁盘操作将该文件读入缓存（可能要丢弃其他一些缓存的文件，以便腾出空间）。从磁盘上读取文件后，将该文件放入缓存，同时把它发送给客户。

这种方案的优点是当一个或多个处理模块因为等待磁盘操作或者网络操作的完成而被阻塞时（因此不占用 CPU 时间），其他模块可以继续处理其他的请求。有了 k 个处理模块，吞吐量可以达到单线程服务器下的 k 倍。当然，当磁盘或者网络成为受限因素时，必须使用多个磁盘或者更快的网络才能获得比单线程模式实质性的性能提高。

现代 Web 服务器所做的不只是接受文件名和返回文件。事实上，每个请求的实际处理过程可能非常复杂。由于这个原因，在许多服务器中，每个处理模块要执行一系列步骤。前端将每个入境请求传递给第一个可用模块，然后该模块根据这个特定请求的需要，执行下列步骤中的某个子集。这些步骤发生在 TCP 连接和任何安全传输机制（例如 SSL/TLS，将在第 8 章介绍）建立之后。

- （1）解析被请求的 Web 页面的名字。
- （2）执行对该页面的访问控制。
- （3）检查缓存。
- （4）从磁盘上获取请求的页面或者运行一个创建页面的程序。
- （5）确定响应中的其余部分（比如要发送的 MIME 类型）。

(6) 把响应返回给客户。

(7) 在服务器的日志中增加一个表项。

第(1)步是必需的,因为入境请求中可能并没有包含文件或者程序的字面意义上的实际名字。它或许包含了需要转换的内置缩写。作为一个简单例子,在 URL `http://www.cs.vu.nl` 中的文件名是空的。在这种情况下,有必要将它扩展到某个默认的文件名,通常是 `index.html`。另一个常用的规则是将 `~user` 映射到 `user` 的 `web` 目录。这些规则可以联合使用。因此本书作者之一(AST)的主页可以通过下面的 URL 到达:

`http://www.cs.vu.nl/~ast/`

即使在特定默认目录下的实际文件名是 `index.html`。

同样地,现代浏览器可以指定配置信息,比如浏览器软件和用户的默认语言(例如意大利语或英语)。这样,如果可能,服务器就能为移动设备选择具有小图片和恰当语言的 Web 页面。一般来说,文件名的扩展并不像它首次出现时那样微不足道,因为对于如何把路径映射到文件目录和程序有各种各样的惯例。

第(2)步检查是否存在与该页面关联的任何访问限制。并非所有的网页都向公众开放。确定客户端是否有权限获取一个页面可能依赖于客户端的身份(例如,给出用户名和密码),或者客户端在 DNS 或 IP 地址空间的位置。例如,一个页面可能被限制于只向公司内部员工开放。如何实现这一点则取决于服务器的设计。例如,对于流行的 Apache 服务器,惯常的做法是把后缀名为 `.htaccess` 的文件放置在被限制访问的页面所在的目录,该文件列出了对页面的访问限制。

第(3)步和第(4)步涉及页面的获取。是否可以从缓存中获取页面取决于处理规则。例如,因为程序每次运行都可能产生不同的结果,因此由正在运行的程序创建的页面并不总是可以被缓存。甚至文件也应该偶尔被检查一下,看看它们的内容是否已经发生改变,以便旧的内容从缓存中删除。如果页面需要运行一个程序,那么还存在一个设置程序参数或输入的问题。这些数据来自请求的路径或请求中的其他部分。

第(5)步确定响应页面内容时有关的其他部分。MIME 类型就是一个例子。它可能来自文件的扩展名、一个文件的前几个字或程序的输出、一个配置文件,以及其他可能的来源。

第(6)步通过网络返回页面。为了提高性能,一个 TCP 连接可以被客户端和服务器的连接并且返回请求的每个响应,以便它与正确的请求关联。

第(7)步是为了行政管理的需要在系统日志中增加一个表项反映此次访问,同时还记录任何其他重要的统计数据。这种日志可以在日后被用来挖掘出有关用户行为的有价值信息,例如,人们访问网页的次序。

Cookie

到目前为止,正如我们所描述的那样,浏览网站只涉及一系列独立页面的获取。完全没有登录会话的概念。浏览器发送一个请求给服务器,并获得返回的文件。然后,服务器彻底忘记它已经看到过哪些特定的客户。

这种模式对于检索公开可用的文件资源完全足够,而且在 Web 创造初期运作良好。然而,它不适合为不同的用户返回不同的页面,应该给用户返回什么页面取决于他们之前在

服务器上做了什么。这样的行为需要与网站进行多次交互。例如，某些网站（例如报纸）要求客户注册（并且有可能要付费）才能使用。这就提出了一个问题，服务器如何区分来自以前注册过的用户的请求还是来自没有注册过的其他人的请求？第二个例子是电子商务。如果用户在电子商店漫步，不时地折腾放到虚拟购物车中的项目，服务器如何保持跟踪其购物车的内容？第三个例子是定制的门户网站，比如雅虎。用户可以设置其个性化的初始页面，该页面只给出他们想要的信息（例如，他们的股票和喜爱的运动队），但是，如果服务器不知道用户是谁，它该如何显示正确的页面？

乍一看，读者可能会立即想到服务器可以通过观察用户的 IP 地址来跟踪他们。然而，这个想法并不奏效。首先，许多用户通过共享的计算机工作，特别是在家里；而且 IP 地址仅仅标识了计算机而不标识用户。其次，更糟糕的是，许多公司使用 NAT，因而所有用户发出的出境数据包具有相同的 IP 地址。也就是说，所有 NAT 盒子后面的计算机从服务器的角度来看是同一台机器。

这个问题的解决方案是采用了一项称为小甜饼（Cookie，它是一小段文本信息，伴随着用户请求页面在 Web 服务器和浏览器之间传递）的备受批评的技术。这个名称来自于很早以前的程序员行话，即程序调用一个过程并得到一些信息，这些信息稍后在完成某些工作时可能需要用得到。从这种意义上来说，UNIX 文件描述符或 Windows 对象句柄都可以被看作是一种 Cookie。Cookie 首次实现在 1994 年 Netscape 浏览器中，后来在 RFC 2109 中对此作了正式定义。

当客户请求一个 Web 页面时，服务器除了提供所请求的页面以外，还以 Cookie 的形式提供了一些附加的信息。Cookie 是一个相当小的命名的串（最多 4 KB），服务器将它与浏览器关联。这种关联与用户关联不一样，但它非常接近而且比 IP 地址更有用。浏览器把服务器所提供的 Cookie 通常存储在客户机磁盘 Cookie 目录下一段时间，这样在整个浏览器调用期间一直坚持 Cookie，除非用户禁用 Cookie。Cookie 只是字符串，而不是可执行程序。原则上，一个 Cookie 可能包含病毒，但由于 Cookie 只被当作数据处理，因而不存在病毒得以实际运行从而造成损害的正式途径。然而，始终有可能存在一些黑客利用一个浏览器的漏洞，来激活病毒。

一个 Cookie 可能包含至多 5 个字段，如图 7-22 所示。域（Domain）指出 Cookie 来自何方。浏览器应该检查服务器没有谎报它们的域名。每个域为每个客户端应该存储不超过 20 个 Cookie。路径（Path）是服务器目录结构中的一个路径，它标识了服务器文件树的哪些部分可能使用该 Cookie。路径通常是/，这意味着整棵树。

域	路径	内容	过期	安全
toms-casino.com	/	CustomerID=297793521	15-10-1017:00	是
jills-store.com	/	Cart=1-00501;1-07031;2-13721	11-1-1114:22	不
aportal.com	/	Prefs=Stk:CSCO+ORCL;Spt:Jets	31-12-2023:59	不
sneaky.com	/	UserID=4627239101	31-12-19 23:59	不

图 7-22 Cookie 的某些例子

内容（Content）字段采用“名字 = 值”的形式。这里名字和值都可以是服务器期望的任何内容。这个字段正是存放 Cookie 内容的地方。

过期时间（Expires）字段指定了该 Cookie 何时过期。如果这个字段不存在，则浏览器

在退出时将丢弃 Cookie，这样的 Cookie 称为非持续 Cookie (nonpersistent cookie)；如果针对某个 Cookie 提供了时间和日期，那么这样的 Cookie 称为持续 Cookie (persistent cookie)，它被一直保存直至过期为止。过期时间采用了格林尼治标准时间。为了从客户的硬盘上删除一个 Cookie，服务器只需将它再发送一次，但是将该 Cookie 的过期时间指定为一个已经过去的时间即可。

最后，通过设置安全 (Secure) 字段指示浏览器只向使用安全传输连接的服务器返回 Cookie，所谓的安全传输就是 SSL/TLS (我们将在第 8 章描述)。这个功能可用于电子商务、银行和其他安全类应用。

我们现在已经看到了如何获得 Cookie，但如何使用 Cookie 呢？在浏览器向某个 Web 站点发出一个页面请求之前，浏览器检查它的 Cookie 目录，确定这个请求前往的目标域是否在当前客户端放置了 Cookie。如果存在相应的 Cookie，则该域放置的所有 Cookie 都被包含到请求消息中。服务器得到了这些 Cookie 以后，就可以按它所期望的方式来解释它们。

我们现在来看 Cookie 的一些可能用法。在图 7-22 中，第一个 Cookie 由 toms-casino.com 设置，并且被用来标识一个顾客。当该顾客下个星期登录进去花掉了一些钱时，浏览器将 Cookie 发送过去，因此服务器就知道他是谁了。有了顾客的 ID，服务器就可以在数据库中查找该顾客的消费记录，并利用这些记录信息来创建一个合适的 Web 页面显示给该顾客。如果已知该顾客爱好赌博，则为其显示的页面可能由扑克游戏、今日赛马比赛名单，或老虎机组成。

第二个 Cookie 来自 joes-store.com。这里的场景是顾客在商店里闲逛，寻找想购买的好东西。当她发现一件便宜货并单击它时，服务器将该货物放入她的购物车 (由服务器维护)，同时创建一个包含商品数量和产品代码的 Cookie，并把它发送给客户。当客户继续在商店里闲逛时，每当客户点击一个新页面，该 Cookie 随着该新页面的请求被返回到服务器端。随着顾客购买的物品累计得越来越多，服务器将它们统统加入到 Cookie 中。最后，当客户单击“前往结账”(PROCEED TO CHECKOUT) 时，这个 Cookie 现在包含了她想购买商品的完整列表，它和当前的请求一起被发送给服务器。通过这种方式，服务器就能够精确地知道顾客想购买哪些东西。

第三个 Cookie 用于 Web 门户网站。当用户单击一个指向该门户的链接时，浏览器将 Cookie 发送过去。它告诉该门户网站创建一个页面，包含 Cisco 公司和 Oracle 公司的股票价格，以及纽约喷气机队 (New York Jets) 橄榄球赛的结果。由于 Cookie 最多可以达到 4 KB 长，所以还有足够的空间来存放关于报纸头条新闻、本地天气、特价信息等更多用户喜好的信息。这样网站经营者可以了解用户如何浏览他们的网站，广告商为一个特定用户建立浏览的广告或网站的概况。备受争议的是用户通常不知道他们的活动被跟踪了，即使详细的资料和跨越看似与网站无关。尽管如此，Web 跟踪 (Web tracking) 是一项大生意。提供并跟踪广告的 DoubleClick 跻身于世界上最繁忙的前 100 网站，这是由 Web 监测公司 Alexa 作出的排名。Google Analytics 为运营商跟踪站点的使用情况，它已被世界上最繁忙的 10 万个网站中一半以上的网站所使用。

利用 Cookie，服务器很容易地就能跟踪用户的活动。假设一个服务器想知道它有多少个只来过一次的访问者，以及每个人在离开该站点前看过多少页面。当第一个请求到来时，它不会随带任何 Cookie，因此服务器返回一个包含“Counter=1”的 Cookie。用户浏览本

站点的后续网页将会把这个 Cookie 送回给服务器。服务器每次递增该计数器并送回给客户。通过跟踪这个计数器，服务器就可以知道有多少人在看了第一个页面后就离开了，有多少人看了第二个页面才离开，等等。

跟踪用户跨越站点的浏览行为仅仅稍微复杂一点。其工作过程如下所述。一家广告代理商，比如说 Sneaky 广告，它联系了一些主要的 Web 站点，在这些站点的页面上放置一些广告来宣传它企业客户的产品，为此它当然向这些站点的所有者支付一定的费用。广告公司并不是向这些站点提供一个放在每个页面上的 GIF 或 JPEG 文件，而是提供一个 URL，这个 URL 被加入到每个页面中。广告公司给出的每个 URL 包含一个路径上的唯一数字，例如

<http://www.sneaky.com/382674902342.gif>

当用户第一次访问一个包含该广告的面 P 时，浏览器获取 HTML 文件。然后浏览器检查该 HTML 文件，并看到了指向 www.sneaky.com 上图像文件的链接，因此它向 www.sneaky.com 发送一个对该图像的请求。一个包含广告 GIF 文件，连同包含唯一用户 ID（即图 7-22 中 4627239101）的 Cookie，一起被返回给客户。Sneaky 公司记录下这样的事实：此 ID 的用户曾经访问过页面 P。要做到这点很容易，因为被请求的路径（382674902342.gif）只在页面 P 上被引用过。当然，实际的广告可能出现在成千上万个页面，但每次都有一个不同的文件名。Sneaky 公司也许在每次发布广告时从产品制造商那里获得几美分的收益。

稍后，当这个用户访问另一个包含 Sneaky 广告的 Web 页面时，浏览器首先从服务器获回 HTML 文件，然后它看到页面上有一个指向 <http://www.sneaky.com/193654919923.gif> 的链接，于是请求该文件。由于浏览器已经有一个来自 [sneaky.com](http://www.sneaky.com) 域的 Cookie，于是它将这个含有用户 ID 的 Sneaky Cookie 也包含在请求中。Sneaky 现在知道该用户已经访问了第二个页面。

在适当的时候，即使用户从未点击过任何广告，Sneaky 也能够建立起有关该用户浏览习惯的详细轮廓。当然，它还没有用户的名字（虽然它有用户的 IP 地址，也许根据其他数据库信息就足以推断出用户的名字）。然而，如果该用户曾经将他的名字提供给任何一个与 Sneaky 合作的站点，那么现在 Sneaky 就拥有了一份带有用户名字的完整轮廓信息，它可以卖给任何一个有意购买此类信息的人。出售这些信息可能会给 Sneaky 带来足够的利润，使它能在更多的 Web 站点上放置更多的广告，从而收集更多的信息。

如果 Sneaky 想要做得更加偷偷地不为人知，则广告不必做成传统的横幅广告。一个由背景颜色单像素组成的“广告”（因此是不可见的）与横幅广告具有完全相同的效果：它要求浏览器取回这个 1×1 像素的 gif 图像，并将该像素所属域的所有 Cookie 发送给服务器。

Cookie 已经成为是否侵害用户在线隐私的一个辩论焦点，因为像上面这样的跟踪行为侵犯了用户的在线隐私。在整个商业中最阴险的环节在于许多用户完全不知道自己的信息被他人收集，甚至可能认为自己是安全的，因为他们没有点击任何广告。出于这个原因，跨站点跟踪用户的 Cookie 被许多人认为是间谍软件（spyware）。看看你的浏览器中是否早就已经存储了 Cookie。大多数浏览器会显示此信息，同时还会显示当前的隐私偏好设置。你或许会惊讶地发现姓名、电子邮件地址或密码以及不透明的标识符。希望你不会发现信用卡号码，但 Cookie 被滥用的潜在性非常明显。

为了维护外表上的隐私，有些用户将他们的浏览器配置成拒绝所有的 Cookie。然而，这会带来一些问题，因为许多 Web 站点没有 Cookie 无法正常工作。另外，大多数浏览器允许用户阻止第三方 Cookie。第三方 Cookie (third-party cookie) 是从一个不同于主页网站的其他网站获得的 Cookie，例如，sneaky.com Cookie 就是在与一个完全不同网站的 P 页面交互时所用的。阻止这些 Cookie 有助于防止跨网站的跟踪。安装浏览器的扩展也可以提供细粒度的控制如何使用 Cookie（或者说，不使用 Cookie）。随着辩论的继续，许多公司正在制定有关隐私政策，限制它们如何共享信息，以便防止用户信息被滥用。当然，政策只是公司简单地说它们将如何处理信息。例如：“我们可能会使用从你那里收集到的信息来指导我们的业务”——这或许正是销售信息。

7.3.2 静态 Web 页面

Web 的基础是将 Web 页面从服务器传输到客户端。在最简单的形式中，Web 页面是静态的。也就是说，它们就是存放于服务器上的文件，每次被客户端获取和显示表现的都是一样的方式。然而，仅仅因为它们是静态的，并不意味着页面在浏览器端是呆滞的。包含一段视频的页面也可以是一个静态 Web 页面。

正如前面提到的，在这个 Web 中，大多数页面以 HTML 语言来编写。教师的网页通常是静态的 HTML 页面，而公司的网页则通常是由一个网页设计公司制作的动态页面。在本节中，我们将简要介绍静态 HTML 页面，这是以后学习的基础。如果读者早已熟悉 HTML，可以直接跳到下一节，在那里我们描述了动态网页内容和 Web 服务。

HTML——超文本标记语言

Web 引入了一种称为超文本标记语言 (HTML, HyperText Markup Language) 的语言。HTML 允许用户生成一个包含了文本、图形和指向其他 Web 页面指针的 Web 页面。HTML 是一种标记语言，或一种描述了如何格式化文档的语言。“标记”这个术语来自过去，那时编辑在要印刷的文档上实际标记出有关信息，告诉印刷工（那时还是人工印刷）应该使用什么字体等。因此，标记语言包含了格式化的显式命令。例如，在 HTML 中，意味着粗体字型的开始，表示粗体字型的结束。LaTeX 和 TeX 是标记语言的其他实例，这些语言为大多数学术作者所知所用。

相对于其他无显式标记的语言来说，标记语言的关键优点是将内容与其应该如何表示相分离。如此一来，编写浏览器就非常简单：浏览器只需理解标记命令并将这些命令应用于内容即可。在每个 HTML 文件中嵌入所有的标记命令，并且标准化这些标记命令，使得任何一个 Web 浏览器都可以读取任何 Web 页面，并对页面重新格式化。这点非常关键，因为一个页面有可能是在一台具有 1600×1200 大小 24 位颜色窗口的高端计算机上被设计出来的，但它必须能在一个移动电话只有 640×320 大小 8 位颜色的窗口中显示。

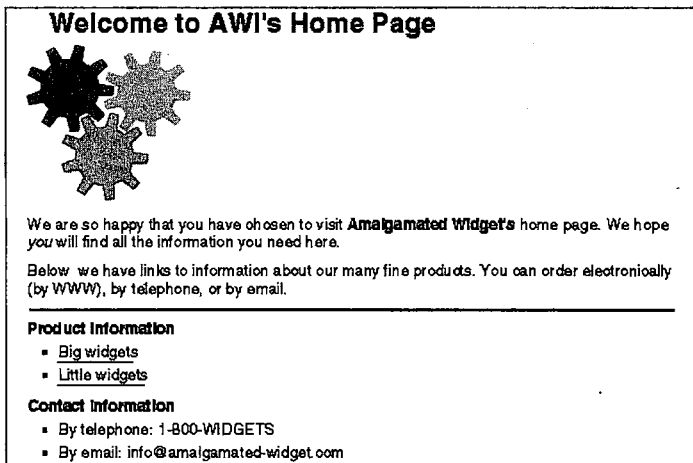
尽管可以用任何一个纯文本编辑器来编写这样的文档，而且确实有许多人也是这样做的，但是，也可以使用文字处理器或者专用的 HTML 编辑器来完成大部分工作（但是，相应地给予用户对最终结果的所有细节的控制能力也有所减弱）。

图 7-23 给出了一个以 HTML 编写的简单 Web 页面及其在浏览器上的显示。一个 Web 页面由一个头部和一个主体组成，它们都被包括在<html>和</html>标签 (tag) 之间，这些

标签就是格式化命令；虽然缺少这两个标签大多数浏览器也不会出错。正如图 7-23(a)所示，头部由<head>和</head>标签括起来，而主体部分则由<body>和</body>标签括起来。标签中的字符串称为指示符（directive）。大多数 HTML 标签（并不是全部）都采用这种格式。即用<something>标注 something 的开始，而用</something>标注 something 的结束。

```
<html>
<head> <title> AMALGAMATED WIDGET, INC. </title> </head>
<body> <h1> Welcome to AWI's Home Page </h1>
 <br>
We are so happy that you have chosen to visit <b> Amalgamated Widget's</b>
home page. We hope <i> you </i> will find all the information you need here.
<p>Below we have links to information about our many fine products.
You can order electronically (by WWW), by telephone, or by email. </p>
<hr>
<h2> Product information </h2>
<ul>
  <li> <a href="http://widget.com/products/big"> Big widgets </a> </li>
  <li> <a href="http://widget.com/products/little"> Little widgets </a> </li>
</ul>
<h2> Contact information </h2>
<ul>
  <li> By telephone: 1-800-WIDGETS </li>
  <li> By email: info@amalgamated-widget.com </li>
</ul>
</body>
</html>
```

(a)



(b)

图 7-23

(a) 一个 Web 页面的 HTML；(b) 格式化后的页面

标签既可以是小写也可以是大写。因此<head>和<HEAD>含义相同，但为了兼容，最好采用小写。HTML 文档的实际布局结构无关紧要。HTML 解析器将忽略额外的空格和回

车，因为它们必须重新对文本进行格式化，以便使得这些文本适合当前的显示区域。因此，可以任意添加空格使得 HTML 文档更具可读性，而大部分空格并不是所需要的。另外一个后果是空行无法真正划分出单独一段，因为它们在显示时被忽略掉了。因此需要一个显式的标签来分段。

有些标签带有（命名的）参数，这些参数称为属性（attribute）。例如，在图 7-23 中，标签

第二个属性给出了在图像无法显示时的替代文字。对于每个标签，HTML 标准给出了一个允许的参数列表，包括参数及其含义。由于每个参数都有名字，因而给定参数的出现顺序就不重要了。

从技术上讲，HTML 文档是用 ISO 8859-1 Latin-1 字符集编写的，但是如果用户的键盘只支持 ASCII 字符，则需要用转义序列来代表特殊的字符，比如 è。标准给出了特殊字符的列表。它们都以“&”符号作为开头，以“;”作为结束。例如， 表示空格、`表示 è、´表示 é。由于<、>和 &有特殊的含义，所以它们只能用转义序列来表示，分别是<、>和 &。

头部中的主要内容是标题（title），它由<title>和</title>来定界。某种特定的元信息也可以出现在这里，虽然在我们的例子中没有给出来。标题本身不显示在页面上。有些浏览器用它来标记页面的窗口。

图 7-23 使用了几种头。每个头由<h>标签生成，其中 n 是一个 1 到 6 之间的数字。因此，<h1>是最重要的标题；<h6>是最不重要的标题。如何在屏幕上突出这些标题是浏览器的责任。通常较小数字的标题会采用较大和较重的字体来显示。浏览器也可能为每种级别的标题选择使用不同的颜色。一般来说，<h1>标题的字体较大较粗，并且在标题的上方和下方至少有一个空行。相对地，<h2>标题较小，且上方和下方只有较少的空白。

标签和<i>分别用来表示进入粗体和斜体模式。<hr>标签强制中断，并画一条横跨屏幕的水平线。

<p>标签开始一段。例如，浏览器可能会显示插入一个空行和一些缩进量。有趣的是，为了标志一个段落结束的</P>常常被懒惰的 HTML 程序员忽略。

HTML 提供了许多机制来构造列表，包括嵌套列表。未排序的列表，例如图 7-23 所示，以开始；表项都以标记。还可以用标签开始一个有序的列表。未排序列表中的个别表项通常使用标签（•）开始；而有序列表中的个别表项由浏览器对其进行编号。

最后，我们讨论超链。图 7-23 中给出的一些超链实例使用了锚<a>和标签。<a>有不同的参数，最重要的是 href，即链接的 URL。<a>和之间的文本是显示用的。如果这段文本被选中，则浏览器跟随超链转到一个新的页面。还可以链接其他元素。例如，在<a>和之间也可以放一个用标签的图像。在这种情况下，显示的是图像，单击图像可以激活该超链接。

在这个简单例子中我们没有看到许多其他的 HTML 标记和属性。例如，<a>标签可以

带一个参数 `name`，该参数设置一个超链接，允许一个超链接指向一个页面的中间。这个参数非常有用，例如，对于那些由可点击表内容启动的 Web 页面。用户通过点击表的内容中的一个表项，就可以跳转到同一页面的相应部分。另一个不同的标记 `<br>`，它迫使浏览器回车并开始新的一行。

也许了解标签的最好办法是看它们如何发挥作用。要做到这一点，你可以选择一个 Web 页面，然后看你浏览器中的 HTML 页面是如何放在一起的。大多数浏览器有一个查看源文件（VIEW SOURCE）的菜单项（或类似的东西）。选择该项目显示当前页面的 HTML 源代码，而不是它的格式化输出。

我们已经勾画出从早期 Web 起就存在的标签的大致轮廓。自此以后，HTML 保持着不断的演进。图 7-24 显示了一些 HTML 后续版本添加的功能。HTML 1.0 指 Web 引入时使用的 HTML 版本。随着 Web 空间的爆炸，在几年时间内 HTML 版本快速连续推出了 2.0、3.0 和 4.0。在 HTML 4.0 之后，几乎用了将近十年，才走到下一个主要版本 HTML 5.0 的标准化。因为它是一个重大的升级，巩固了浏览器处理丰富内容的方式，因此 HTML 5.0 的努力还在进行着，预计最早在 2012 年以前不会产生新的标准。不管标准如何，主流浏览器在功能上早已支持 HTML 5.0。

项目	HTML 1.0	HTML 2.0	HTML 3.0	HTML 4.0	HTML 5.0
超链接	×	×	×	×	×
图像	×	×	×	×	×
列表	×	×	×	×	×
活动地图和图像		×	×	×	×
表单		×	×	×	×
方程式			×	×	×
工具条			×	×	×
表格			×	×	×
访问功能				×	×
对象嵌入				×	×
风格				×	×
样式表				×	×
脚本				×	×
视频和音频				×	×
内联矢量图型					×
XML 表示					×
后台线程					×
浏览器存储					×
画曲线					×

图 7-24 HTML 版本之间的某些差异

整个 HTML 版本的进展都是关于加入新的功能。这些功能是人们所希望有的，但在成为标准化之前不得不以非标准的方式来处理（比如，插件）。例如，HTML 1.0 和 HTML 2.0 没有表格功能，表格功能在 HTML 3.0 才被添加进来。一个 HTML 表中包含一个或多个行，每行包含一个或多个表单元（table cell），这些单元可以包含更广泛的素材（比如，文本、图像和其他表）。在 HTML 3.0 之前，需要一张表的作者不得不诉诸特殊方法来达到表格效果，诸如包括一幅显示表信息的图像。

在 HTML 4.0 中，增加了更多的新功能。这些功能包括残障用户的访问功能、对象嵌入（由标签生成，因此其他对象也可以被嵌入到页面中）、支持脚本语言（允许显示动态的内容）以及更多的辅助功能。

HTML 5.0 包含了许多功能来处理富媒体，即现在经常在网络上使用的丰富媒体。页面可以包括视频和音频，并且浏览器可以正常播放而无须用户安装插件。画画作为矢量图形可内置在浏览器中，而不必使用位图图像格式（如 JPEG 和 GIF）。还有更多支持在浏览器中运行脚本，诸如计算和存储访问的后台线程。所有这些功能有助于支持 Web 页面比文档看起来更像一个具有用户接口的传统应用。这是 Web 发展的方向。

输入和表单

目前我们尚未讨论的一个重要功能是输入。HTML 1.0 基本上是单向的。用户可以从信息提供者获取网页，但很难在另一个方向将信息发送回去。很快，双向通信的需求变得很明显，比如允许通过网页提交产品订货单、在线填写注册卡、输入搜索的词汇等。

将用户的输入发送到服务器（通过浏览器）需要两种支持。首先，它要求 HTTP 可以在这个方向携带数据。我们在本节后面描述这是怎么做的：它使用了 POST 方法。第二个要求是能够提出用户界面元素，用来收集和封装输入的数据。HTML 2.0 具备这种功能，并包括了表单。

表单上包含框或者按钮，允许用户填写信息或做出选择，然后将信息发送回页面的所有者。正如在图 7-25 的例子中看到的那样，表单的编写就像 HTML 的其他部分。请注意，表单仍然是静态内容。无论谁使用它们，它们都表现出相同的行为。我们将在后面涉及的动态内容提供了更复杂的方法来收集用户输入，具体方法是发送一个其行为可能依赖于浏览器环境的程序。

像所有的表单一样，这个表单也是被包含在<form>和</form>标签之间。这个标签的属性说明了针对数据输入应该做什么，在这种情况下，使用 POST 方法将数据发送到指定的 URL。没有被包括在标签之内的文本只是用来显示的。一个表单中允许出现所有常见的标签（例如，），以便让页面的作者控制表格在屏幕上的外观。

这张表使用了三种输入框，每个输入框使用了<input>标签。它有各种参数来确定显示框的大小、性质和用法。最常见的形式是空白字段，用来接受用户输入的文本；框被检查之后，通过按下 submit（提交）按钮把用户输入的数据返回给服务器。

第一种输入框是 text（文本）框，出现在文本“Name”之后。这个框有 46 个字符宽并期待用户输入一个字符串，然后这个字符串被存储在 customer 变量中。

表单的下一行要求输入用户的街道地址，它宽 40 个字符。接着下面一行是城市、州和国家。在这些字段之间没有使用<p>标签，因此，如果放得下，浏览器会在一行上（而不是作为分开的一段段）显示这些字段。至于浏览器，这段包含了六个表项：三个字符串与三个框交替出现。接下来一行要求输入信用卡号和失效日期。只有当采取了足够的安全措施以后，方才可以在 Internet 上传输信用卡号。我们将在第 8 章中讨论相关的安全话题。

在失效日期之后我们遇到了一个新的特性：单选按钮（radio button）。当用户必须在两个或者更多个选项中选择其中之一时，需要用到单选按钮。这里的智力模型像带有六个选台按钮的汽车收音机。点击其中一个按钮的同时关掉了同组中的其他按钮。按钮的视觉表

```

<html>
<head> <title> AWI CUSTOMER ORDERING FORM </title> </head>
<body>
<h1> Widget Order Form </h1>
<form ACTION="http://widget.com/cgi-bin/order.cgi" method=POST>
<p> Name <input name="customer" size=46> </p>
<p> Street address <input name="address" size=40> </p>
<p> City <input name="city" size=20> State <input name="state" size =4>
Country <input name="country" size=10> </p>
<p> Credit card # <input name="cardno" size=10>
Expires <input name="expires" size=4>
M/C <input name="cc" type=radio value="mastercard">
VISA <input name="cc" type=radio value="visacard"> </p>
<p> Widget size Big <input name="product" type=radio value="expensive">
Little <input name="product" type=radio value="cheap">
Ship by express courier <input name="express" type=checkbox> </p>
<p><input type=submit value="Submit order"> </p>
Thank you for ordering an AWI widget, the best widget money can buy!
</form>
</body>
</html>

```

(a)

(b)

图 7-25

(a) 一张订单的 HTML; (b) 格式化后的页面

现由浏览器决定。构件大小 (Widget Size) 也使用了两个单选按钮。这两组按钮由它们的 name 字段来区分, 而不是通过使用像 `<radiobutton> ... </radiobutton>` 这样的标签来划定静态范围。

Value (值) 参数用来指示按下了哪个单选按钮。例如, 根据用户选择的是哪个信用卡按钮, 变量 cc 将被设置为字符串 “mastercard” 或者 “visacard”。

在两组单选按钮之后我们来到发货选项, 它用复选框 (checkbox) 表示。复选框既可以被选中, 也可以未被选中。单选按钮必须选中一个, 而与此不同的是每个复选框都可以被选中也可以未被选中, 相互之间完全独立。

最后, 我们来到 submit (提交) 按钮。value 字符串就是按钮的标签, 并且显示在按钮上。当用户点击 submit 按钮时, 浏览器将收集到的信息打包成一个很长的行, 并将其发送回服务器; 该服务器由作为 `<form>` 标签一部分的 URL 提供。发送时要用到一个简单的编码。“&” 用来分隔字段, “+” 用来表示空格。对于我们示例的表单, 发送的行可能看起来像

图 7-26 所示的内容。

```
customer=John+Doe&address=100+Main+St.&city=White+Plains&
state=NY&country=USA&cardno=1234567890&expires=6/14&cc=mastercard&
product=cheap&express=on
```

图 7-26 浏览器对服务器的一种可能响应，包括了用户填入的信息

该字符串作为一行被发送回服务器（这里因为页面不够宽，它被分成了三行）。只有服务器能理解这个串的意义，它最有可能做的是将信息传递到一个处理这个字符串的程序。我们将在下一节讨论如何做到这点。

还可以有其他类型的输入，在这个简单的例子中没有显示出来。两种其他类型的输入是 password（密码）和 textarea（文本区域）。password 框与 text 框（这是不需要命名的默认类型）相同，只有一点除外，那就是在输入字符时并不显示出来。textarea 框也与文本框相同，但是它可以包含多个行。

对于那些必须从中作出选择的长列表，可以用提供的<select>和</SELECT>标签将一系列替代品清单包括起来。这个列表经常呈现为一个下拉菜单。下拉菜单的语义就是那些单选按钮的语义，除非给出多个参数，在这种情况下语义变成复选框的语义。

最后，还有一些方法可用来表示默认或初始值，这些值用户是可以改变的。例如，如果一个 text 框给出了一个 value 字段，那么表单中的内容会显示出来，以便用户编辑或删除。

CSS——层叠样式表

HTML 的最初目标是指定文档的结构，而不是文档的外观。例如：

```
<h1> Deborah's Photos </h1>
```

指示浏览器要强调标题，但没有说明有关字型、点的大小或颜色等任何信息。标题的外观如何表现留给浏览器考虑，因为浏览器应该知道显示器的属性（例如，它有多少像素）。然而，许多网页设计师希望能绝对控制自己设计的网页如何呈现，因此新的标签被添加到 HTML 以便控制页面的外观，例如：

```
<font face="helvetica" size="24" color="red"> Deborah's Photos </font>
```

此外，还添加了在屏幕上准确定位的控制方法。这种方法的麻烦在于它既乏味，又产生不可移植的臃肿 HTML。虽然一个页面可能在创建它的浏览器上表现非常完美，但它可能在另一个浏览器、其他同一浏览器的不同版本下或在不同屏幕分辨率的显示器上表现得一塌糊涂。

一种更好的替代方案是使用样式表（style sheet）。文本编辑器中的样式表允许作者将文本与逻辑风格关联，而不是与物理风格相关，例如，“初始段”而不是“斜体文本”。每种风格的外观单独定义。通过这种方式，如果作者决定改变以蓝色 14 点斜体表示的初始段用令人震惊的粉红色粗体 18 点表示，那么所需要做的只是改变转换整个文档的一个定义。

层叠样式表（CSS，Cascading Style Sheets）将样式表引入到 HTML 4.0 的 Web，虽然在 2000 年以前它没有被广泛使用，也没有得到浏览器的广泛支持。CSS 定义了一种简单的

语言，用来描述控制标签内容外观的规则。让我们来看一个例子。假设，AWI 希望时髦的网页用乳白色的背景，以 Arial 字体显示海军文本，而且每个级别的标题将文本分别额外放大 100% 和 50%。图 7-27 的 CSS 定义给出了这些规则。

```
body {background-color:linen; color:navy; font-family:Arial;}
h1 {font-size:200%;}
h2 {font-size:150%;}
```

图 7-27 一个 CSS 例子

可以看出样式的定义很紧凑。每一行选择一个它适用的元素并且给出属性的值。一个元素的属性作为默认值适用于所有它包含的其他 HTML 元素。因此，body 的样式设置了主体中文本段落的样式。也有方便速记的颜色名称（例如，red）。任何没有定义的样式都以浏览器的默认值填充。这种行为使得样式表的定义成为可选，没有定义也可以产生某种合理的表现。

样式表可以被放置在 HTML 文件中（例如，使用<style>标签），但更常见的做法是将它们放置在一个单独的文件中，然后引用它们。例如，AWI 页面的<head>标签可以修改成引用文件 awistyle.css 中的样式表，如图 7-28 所示。这个例子还显示了 CSS 文件的 MIME 类型是 text/css。

```
<head>
<title> AMALGAMATED WIDGET, INC. </title>
<link rel="stylesheet" type="text/css" href="awistyle.css" />
</head>
```

图 7-28 包括了一个 CSS 样式表

这种策略有两大优势。首先，它可以使得一组样式被应用到一个网站上的许多网页。这种组织方式带来了页面外观上的一致性，即使这些页面在不同的时间由不同的作者开发；而且允许通过编辑一个 CSS 文件而不是 HTML 来改变整个网站的外观。这个方法可以与 C 程序中的 #include 文件相类比：在这里改变一个宏的定义就等效于改变了所有包括该头文件的程序。第二个好处是下载的 HTML 文件保持得很小。这是因为浏览器可以下载 CSS 文件的一个副本，供所有引用它的页面使用。浏览器并不需要为每个网页的定义下载一份新副本。

7.3.3 动态 Web 页面和 Web 应用

到目前为止我们使用的静态页面模型将页面作为多媒体文档处理，这些文档被方便地链接在一起。这是 Web 初期的一个拟合模型，因为大量的信息都可在线查询。如今，围绕着 Web 的许多兴奋点在于可将它用作应用程序和服务。应用例子包括在电子商务网站购买产品、检索图书馆目录、探索地图、阅读和发送电子邮件，以及进行文档合作。

这些新的用途犹如传统的应用软件（例如，邮件阅读器和文字处理器）。扭曲的只是这些应用程序运行在浏览器内部，而用户数据存储在互联网数据中心的服务器上。它们利用 Web 协议通过 Internet 访问信息，而浏览器显示用户界面。这种方法的优点是用户不需要安装单独的应用程序，可以从不同的计算机访问用户数据，而且由服务提供商负

责备份。它被证明是如此地成功，几乎可以和传统的应用软件相媲美。当然，这些应用程序由大型服务提供商免费提供的事实也有助于推动应用的进一步展开。这种模式就是云计算（cloud computing）的普遍形式，它将计算从个人台式电脑转移到 Internet 上的共享服务器集群。

为了运行应用程序，Web 页面不能再是静态的了，动态内容是必需的。例如，一个图书馆目录的页面应该反映出哪些书籍当前可借和哪些书籍已经借出因而不可用等书籍的当前状态。类似地，一个有用的股票市场页面将允许用户与页面交互，以便查看不同时期的股票价格，以及计算利润和亏损。这些事例表明，动态内容可以由服务器上或浏览器内（或者同时运行在两个地方）运行的程序产生。

在本节中，我们将依次考查这两种情况。一般的情况如图 7-29 所示。例如，考虑一个地图服务：让用户输入一个街道地址，然后给出相应位置的地图。给定一个位置请求，Web 服务器必须使用一个程序来创建一个页面，该页面显示了该街道地址在地图上的对应位置，而位置信息则是从一个街道数据库和其他地理信息数据库中提取出来的。这一系列动作反映在图中的第 1~3 步。请求（第 1 步）导致在服务器上运行一个程序；该程序查询一个数据库以便生成相应的页面（第 2 步），并将该页面返回给浏览器（第 3 步）。

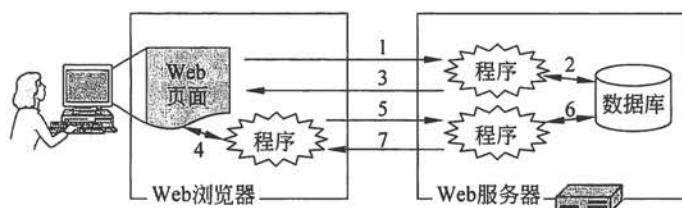


图 7-29 动态页面

然而，有更多的动态内容。返回的页面本身还可能包含着在浏览器中运行的程序。以我们的地图为例，该程序将允许用户发现一条路线，以及不同详细程度地探索周围的附近区域。它会更新页面，跟随用户的指示缩小或放大（第 4 步）。为了处理一些交互事件，该程序可能需要从服务器端获取更多的数据。在这种情况下，程序将给服务器发送一个请求（第 5 步），服务器从数据库中检索出更多的信息（第 6 步），并且给浏览器返回一个响应（第 7 步）。然后，该程序将继续更新页面（第 4 步）。请求和响应都发生在后台，用户甚至可能根本意识不到它们的存在，因为页面的 URL 和标题通常不会改变。通过引入客户端程序，页面可以提供一个比单靠服务器端程序更能响应用户的接口。

服务器端动态 Web 页面生成

让我们更详细地来看服务器端是如何生成动态页面内容的。服务器端必须处理的一个简单情况是表单的使用。回到图 7-25 的例子中，考虑用户填写了 AWI 邮购表单，如图 7-25 (b) 所示，然后点击“提交订单”按钮。当用户点击该按钮后，一个请求就被发送到由 URL 指定的服务器，请求包含表单以及用户填写的表单内容（在这种情况下，是一个到 <http://widget.com/cgi-bin/order.cgi> 的 POST）。这些数据必须由某个程序或脚本来处理。因此，该 URL 指定了要运行的程序，提供的数据作为该程序的输入。在这种情况下，处理将涉及在 AWI 的内部系统输入订单、更新客户记录和收取信用卡的付费。这个请求返回的页面将

取决于处理过程中发生的事情，它并不像一个静态页面那样固定不变。如果订单成功，返回的页面可能给出了预计的出货日期；如果订单不成功，返回的页面可能会给出订购的部件缺货或出于某种原因信用卡不再有效等有关信息。

究竟如何在服务器上运行一个程序而不是检索一个文件则取决于 Web 服务器的设计。Web 协议本身没有说明。这是因为该接口可以是专有的，浏览器并不需要知道里面的细节。至于浏览器，它仅仅负责发出请求和获取页面。

然而，为了让 Web 服务器能够调用程序，已经开发出了标准的 API。这些接口的存在使得开发人员更加容易地把不同的服务器扩展到 Web 应用程序。我们将通过简要地考查两个 API，让你感觉它们是如何工作的。

第一种处理动态页面请求的方法称为公共网关接口 (CGI, Common Gateway Interface)，由 RFC 3875 定义，这是自从有了 Web 就一直可用的方法。CGI 提供了一个接口，允许 Web 服务器与后端程序及脚本通信；这些后端程序和脚本接受输入信息（例如，来自表单），并生成 HTML 页面作为响应。通常，这些后端程序可以用任何一种开发者便利的语言编写，通常都是些方便开发的脚本语言。可以选择 Python、Ruby、Perl 或任何你喜欢的语言。

按照惯例，通过 CGI 调用的程序常驻在一个称为 `cgi-bin` 的目录下，该目录在 URL 中可以看到。服务器将一个请求映射到这个目录下一个程序名，并且以一个单独的进程执行该程序。它把与请求一起发送过来的任何数据作为程序的输入，而程序的输出则提供了一个返回给浏览器的网页。

在我们的例子中，调用的程序是 `order.cgi`，它以图 7-26 所示形式的编码作为输入。它解析参数并处理订单。一个有用的约定是如果没有提供表单的输入数据，那么该程序将返回订单的 HTML。通过这种方式，程序一定要知道表单的表示。

我们将看到的第二种常见的 API 方法相当不同。这里的方法是在 HTML 页面中嵌入少量的脚本，然后让服务器来执行这些脚本以便生成最终发送给客户的页面。编写这些脚本的一种流行语言是超文本预处理器 (PHP, Hypertext Preprocessor)。为了使用 PHP，服务器必须能够理解 PHP，就好像浏览器必须理解 CSS 才可以解释具有样式表的 Web 页面一样。通常，服务器用文件扩展名 `php` 来标识包含 PHP 的 Web 页面，而不是用 `html` 或者 `htm` 扩展名。

PHP 的使用类似于 CGI。作为一个展示其如何处理表单的例子，我们看图 7-30(a) 所示的例子。此图的顶部包含了一个含有简单表单的普通 HTML 页面。这时，`<form>` 标签指明了当用户提交表单后应该调用 `action.php` 文件来处理参数。该页面显示了两个文本框，一个要求输入名字，另一个要求输入年龄。当用户填写了这两个框并且表单被提交后，服务器解析类似图 7-26 那样的字符串，并且把名字放到 `name` 变量中，把年龄放到 `age` 变量中；然后，作为应答，服务器开始处理 `action.php` 文件，如图 7-30(b) 所示。在处理这个文件的过程中，要执行 PHP 命令。如果用户在框中输入的是“Barbara”和“32”，那么返回的 HTML 文件将如图 7-30(c) 所示。因此，使用 PHP 以后处理表单变得非常简单。

尽管 PHP 非常易于使用，但它实际上是一种功能强大的程序设计语言，是 Web 和服务器数据库的接口。它具有变量、字符串、数组，以及在 C 语言中可以找得到的绝大多数控制结构，而且它还有比 `printf` 更强大的 I/O 功能。PHP 是开放源码，可以免费获取，而

```

<html>
<body>
<form action="action.php" method="post">
<p> Please enter your name: <input type="text" name="name"> </p>
<p> Please enter your age: <input type="text" name="age"> </p>
<input type="submit">
</form>
</body>
</html>

```

(a)

```

<html>
<body>
<h1> Reply: </h1>
Hello <?php echo $name; ?>.
Prediction: next year you will be <?php echo $age + 1; ?>
</body>
</html>

```

(b)

```

<html>
<body>
<h1> Reply: </h1>
Hello Barbara.
Prediction: next year you will be 33
</body>
</html>

```

(c)

图 7-30

(a) 包含了一个表单的 Web 页面；(b) 处理该表单输出的 PHP 脚本；
(c) 当输入分别是“Barbara”和“32”时 PHP 脚本的输出

且用途广泛。经过精心设计，PHP 能与 Apache 服务器很好地协同工作；而 Apache 也是开放源码的，是世界上使用最为广泛的 Web 服务器。要了解 PHP 的更多信息，请参看(Valade, 2009)。

我们现在已经看到了两种用来生成动态 HTML 页面的不同方法：CGI 脚本和内嵌的 PHP。除此之外，还存在几种其他技术可选。**Java 服务器页面**（JSP, JavaServer Pages）与 PHP 非常相似，只不过页面中的动态部分是用 Java 编程语言而不是 PHP 编写的。使用这种技术的页面具有文件扩展名.jsp。**活动服务器页面**（ASP.NET, Active Server Page）是 Microsoft 版本的 PHP 和 JSP。它使用 Microsoft 专用的 .NET 网络应用框架来生成动态内容。使用这种技术的页面具有文件扩展名.aspx。在这 3 种技术中，究竟选择哪种通常更多地与策略有关（开放源码与 Microsoft）而并非取决于技术，因为这 3 种语言旗鼓相当。

客户端动态 Web 页面生成

PHP 和 CGI 脚本解决了处理表单输入以及服务器与数据库的交互问题。它们都可以接受来自表单的入境信息、查询一个或多个数据库，然后利用查找结果生成 HTML 页面。它们所不能做的是响应鼠标移动事件，或者直接与用户交互。为了达到这个目的，有必要在 HTML 页面中嵌入脚本，而且这些脚本必须运行在客户机上而不是在服务器上。从 HTML

4.0 开始, 可以通过<script>标签来启用这样的脚本。用来产生这种交互式 Web 页面的技术统称为动态 HTML (dynamic HTML)。

最流行的客户端脚本语言是 JavaScript, 我们现在快速简短地了解它。无论名字看起来有多么的类似, JavaScript 与 Java 编程语言几乎没有共同之处。与其他脚本语言一样, JavaScript 是一种非常高级的语言。例如, 仅仅只要一行 JavaScript 代码就可以弹出一个对话框、等待文本输入, 然后将结果字符串存放在一个变量中。这种高级特性使得 JavaScript 非常适合于设计交互式 Web 页面。另一方面, 它的变异速度比用 X 光机捕捉到的果蝇还要快, 所以, 要想编写出能在所有平台上都良好工作的 JavaScript 程序非常困难, 但也许有一天它会稳定下来。

作为 JavaScript 程序的一个例子, 考虑图 7-31。与图 7-30 类似, 它显示了一个表单要求输入名字和年龄, 并且预测这个人明年的年龄。主体部分基本上与 PHP 例子相同, 主要的区别在于 submit 按钮的声明, 以及其中的赋值语句。赋值语句告诉浏览器, 当按钮被单击时调用 response 脚本, 并且将表单作为一个参数传递给它。

```
<html>
<head>
<script language="javascript" type="text/javascript">
function response(test#form) {
    var person = test#form.name.value;
    var years = eval(test#form.age.value) + 1;
    document.open();
    document.writeln("<html> <body>");
    document.writeln("Hello " + person + ".<br>");
    document.writeln("Prediction: next year you will be " + years + ".");
    document.writeln("</body> </html>");
    document.close();
}
</script>
</head>
<body>
<form>
Please enter your name: <input type="text" name="name">
<p>
Please enter your age: <input type="text" name="age">
<p>
<input type="button" value="submit" onclick="response(this.form)">
</form>
</body>
</html>
```

图 7-31 使用 JavaScript 处理表单

这里全新的部分在于 HTML 文件头部对 JavaScript 函数 response 的声明, HTML 文件的头部通常是标题、背景颜色等保留的。这个函数从表单的 name 字段提取出相应的值, 并将它作为一个字符串存放在 person 变量中。它还提取出 age 字段的值, 并且通过 eval 函数将该值转化为一个整数, 然后加 1 并把结果存放在 years 变量中。接着, 它打开一个用于输出的文档, 利用 writeln 方法向这个文档做了四次写操作, 然后关闭文档。该文档是一个 HTML 文件, 这可以从文档中的各种 HTML 标签看出来。然后浏览器在屏幕上显示这

个文档。

PHP 和 JavaScript 看上去很相似，它们都是嵌入在 HTML 文件中的代码，但它们的处理方式完全不同，理解这一点非常重要。在图 7-30 的 PHP 例子中，当用户单击了 submit 按钮后，浏览器将表单中的信息收集到一个长字符串中，然后将它发送给服务器请求一个 PHP 页面。服务器加载该 PHP 文件，并且执行内嵌的 PHP 脚本，由此产生一个新的 HTML 页面。然后该页面被送回给浏览器以便显示出来。浏览器不能确定这是由一个程序生成的。这个过程由图 7-32(a)的第 1~4 步表示。

在图 7-31 的 JavaScript 示例中，当用户点击 submit 按钮，浏览器就解释该页面中包含的一个 JavaScript 函数。所有的工作都在本地完成，即在浏览器内部完成。这时没有与服务器取得任何联系。这个处理过程如图 7-32(b)的第 1 步和第 2 步所示。因此，结果几乎在瞬间就显示出来，而用 PHP 生成的 HTML 在到达客户端之前可能有几秒钟的延迟。

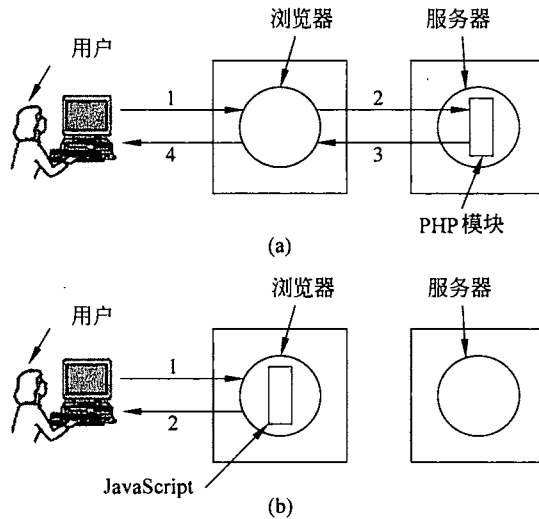


图 7-32
(a) 使用 PHP 的服务器端脚本；(b) 使用 JavaScript 的客户端脚本

这种区别并不意味着 JavaScript 比 PHP 更好。它们的用途完全不同。PHP（同时也暗示着 JSP 和 ASP）用在需要与远程数据库进行交互时；而 JavaScript（和其他我们将要提及的客户端语言，比如 VBScript）则用在需要客户端计算机与用户交互时。当然有可能将两者结合起来，正如我们马上要看的。

JavaScript 并不是使 Web 页面具备高度交互性的唯一方法。另外一种适用于 Windows 平台的方法是 VBScript，它基于 VisualBasic。还有另一种跨平台的流行方法是使用小程序 (Applet)。这些 Java 小程序已经被编译成一种虚拟机的机器指令，这种虚拟机称为 Java 虚拟机 (JVM, Java Virtual Machine)。Applet 可以被嵌入到 HTML 页面中（在<applet>和</applet>之间），并被具有 JVM 能力的浏览器解释执行。因为 Java 小程序是被解释执行而不是被直接执行的，因此 Java 解释器可以防止它们做坏事。至少理论上是这样的。实际上，小程序编写者已经发现在 Java I/O 库中有近乎无限的错误需要被发掘。

Microsoft 对于 Sun 公司 Java 小程序的回应是允许 Web 页面包含 ActiveX 控件，这是指一种被编译成 x86 机器指令的程序，可以直接在硬件裸机上执行。这种特性使得 ActiveX

控件比解释执行的 Java 小程序更快更灵活,因为它可以完成任何普通程序能够做到的事情。当 Internet Explorer 浏览器在某个 Web 页面中发现了一个 ActiveX 控件时,它会下载该控件,检验它的身份然后执行该控件。然而,下载和运行外部程序将会带来巨大的安全问题,我们将在第 8 章中讨论这些安全问题。

由于几乎所有的浏览器都可以解释 Java 程序和 JavaScript,所以如果设计者希望制作出具有高度交互性的 Web 页面,则他至少有两种技术可供选择;另外,如果多平台的移植性不是一个问题,则还可以考虑使用 ActiveX 技术。作为一般规则,JavaScript 比较容易编写,Java 小程序执行得更快一些,而 ActiveX 控件是运行速度最快的。而且,由于所有的浏览器实现了完全相同的 JVM,但是没有两个浏览器实现了相同版本的 JavaScript,所以 Java 小程序比 JavaScript 程序具有更好的移植性。要想了解关于 JavaScript 的更多信息,有很多书可以参考,每一本都有很多页(通常超过 1000 页),例如(Flanagan, 2010)。

AJAX——异步 JavaScript 和 XML

引人注目的 Web 应用程序需要用户界面具备可响应能力,而且能无缝访问存储在远程 Web 服务器上的数据。客户端上的脚本(例如,使用 JavaScript)和服务端上的脚本(比如 PHP)只是提供某种解决方案的基本技术,这些技术通常与其他几个关键技术结合起来一起使用,这些技术的组合就称为异步 JavaScript 和 XML(AJAX, Asynchronous Javascript and Xml)。许多全功能的 Web 应用,比如谷歌的 Gmail、地图和文档都是以 AJAX 编写的。

AJAX 有点混乱,因为它不是一种语言。它是一组需要一起协同工作的技术,正是这些技术使得 Web 应用程序和传统的桌面应用一样能响应用户的每个动作。这些技术包括:

- (1) 用来表现页面信息的 HTML 和 CSS。
- (2) 浏览时改变部分页面的 DOM (Document Object Model)。
- (3) 使得程序与服务器交换应用数据的 XML (eXtensible Markup Language)。
- (4) 程序发送和检索 XML 数据的异步方式。
- (5) 将所有功能组合在一起的 JavaScript。

由于这是一个相当大的集合,我们将逐个考查每一块的属性。我们已经看过了 HTML 和 CSS。它们标准化了如何描述内容和如何显示内容。可以产生 HTML 和 CSS 的任何程序都可以使用 Web 浏览器作为显示引擎。

文档对象模型 (DOM, Document Object Model) 是通过程序访问 HTML 页面的一种表示。这种表示结构化成一棵反映 HTML 元素的树。例如,图 7-30(a)中 HTML 对应的 DOM 树如图 7-33 所示。根是一个 HTML 元素,代表整个 HTML 块。这个元素是 body 元素的父节点, body 元素反过来又是 form 元素的父节点。表单有两个属性,画在表单的右侧,一个属性为表单的方法 (POST), 另一个属性为表单的动作 (请求的 URL)。这个元素有三个子结点,反映了表单包含的两个段落标签和一个输入标签。在树的底部是叶结点,包含了元素或者文字,如文本字符串。

DOM 模型的意义在于它给程序提供了一种改变部分页面内容的简单方法。没有必要重写整个页面。只有包含了需要改变内容的树结点才对页面内容进行替换。当页面内容发生变化时,浏览器将相应地更新显示内容。例如,如果部分页面上的一个图像被 DOM 改变了,浏览器将更新该图像,而不改变页面的其他部分内容。当图 7-31 所示的 JavaScript 示

例被添加到 document 元素中，导致新的一行文本出现在浏览器窗口的底部，我们已经看到上述行为反映在 DOM 动作中。DOM 是一个强大的制作网页方法，并且可以演变发展。

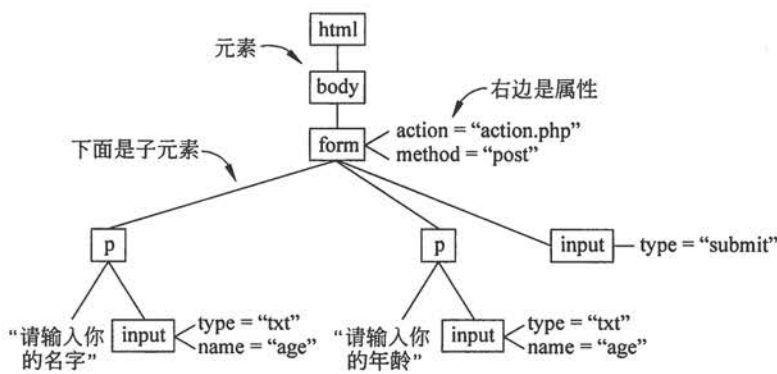


图 7-33 如图 7-30(a)所示 HTML 的 DOM 树

第三个技术，可扩展标记语言（XML，eXtensible Markup Language）是一个说明结构化内容的语言。HTML 将内容与格式混合在一起，因为它关注的是信息的表示。然而，随着 Web 应用程序变得越来越普遍，将独立的结构化内容从它的表示中分离开来的需求也越来越强烈。例如，考虑一个程序，针对一些书它去搜索 Web，以期找出最好的价格。为此它需要分析许多 Web 页面，寻找表项的标题和价格。对程序来说，要找出 HTML 网页中哪里是标题、哪里是价格是一件非常困难的事情。

出于这个原因，W3C 开发了 XML (Bray 等，2006)，为了能够自动处理允许网页内容结构化。与 HTML 不同，XML 没有定义任何标签。每个用户可以定义她自己的标签。图 7-34 给出了一个简单的 XML 文档例子。它定义了一个名为 book_list 的结构，这是一个图书列表。每本书有 3 个字段：标题、作者、出版年份。这些结构非常简单。结构允许有重复的字段（例如，多个作者）、可选字段（例如，有声读物 URL）和替代字段（例如，如果它是印刷版，给出一家书店的 URL；或者如果它是绝版书，给出拍卖网站的 URL）。

在这里例子中，三个字段中的每一个都是不可分割的整体，但它允许被进一步划分。例如，作者字段可作如下划分，以便对搜索和格式化做细粒度的控制：

```
<author>
<first_name> George </first_name>
<last_name> Zipf </last_name>
</author>
```

每个字段进一步划分成子字段和子字段，可以达到任意深度。

图 7-34 全部文件所做的只是定义了一个包含 3 本书的图书清单。它非常适合于浏览器端运行的程序和服务器传输信息，但它没有说明如何显示作为网页的文档。要做到这一点，一个分析信息并判定 1949 是图书出版年的程序可能输出这样的 HTML：标题被标记为斜体文本。另外，一种称为可扩展样式表语言转换（XSLT，eXtensible Stylesheet Language Transformations）的语言可以用来定义如何将 XML 转换成 HTML。XSLT 类似于 CSS，但比 CSS 更强大。我们将竭尽全力为你提供一些细节。

```
<?xml version="1.0" ?>
<book#list>
  <book>
    <title> Human Behavior and the Principle of Least Effort </title>
    <author> George Zipf </author>
    <year> 1949 </year>
  </book>
  <book>
    <title> The Mathematical Theory of Communication </title>
    <author> Claude E. Shannon </author>
    <author> Warren Weaver </author>
    <year> 1949 </year>
  </book>
  <book>
    <title> Nineteen Eighty-Four </title>
    <author> George Orwell </author>
    <year> 1949 </year>
  </book>
</book#list>
```

图 7-34 一个简单的 XML 文档

不用 HTML 而采用 XML 来表示数据的另一个优点是数据更易于被程序分析。HTML 最初是手工编写（而且现在往往仍然如此），所以它多少有点马虎。有时候会忘记写上结束标签，比如<p>。其他标签没有匹配的结束标签，像
。还有其他标签可能被不恰当地嵌套着，而且标签和属性名可以有所不同。大多数浏览器都各尽其能地工作。XML 在定义上更严格也更清晰。标签名字和属性总是小写的，标签必须始终以它们打开的相反次序关闭（或者清楚地表明它们是否是一个没有相应关闭的空标签），而且属性值必须用引号括起来。这种精确定义使得程序解析更加容易，而且绝无二义性。

甚至 HTML 也可以按照 XML 术语来定义。这种方法称为扩展超文本标记语言（XHTML，eXtended HyperText Markup Language）。基本上，它是一个非常挑剔的 HTML 版本。XHTML 页面必须严格符合 XML 规则，否则它们不会被浏览器接受。这种情况下，没有更多伪劣的网页，而且也不存在浏览器之间的不一致。就 XML 而言，其目的是为了产生更好的网页供程序处理（即 Web 应用程序）。虽然 XHTML 自 1998 年就已经出现，但它的发展一直很缓慢。生成 HTML 的人们不明白为什么他们需要 XHTML，而且浏览器的支持也已经落后。现在 HTML 5.0 已经被定义出来，因此页面可以表示成 HTML 也可以表示成 XHTML，这样有助于过渡。最终，XHTML 将取代 HTML，但在完成过渡之前将是一个漫长的过程。XML 也被证明可用作程序之间的流行通信语言。当这种通信由 HTTP 协议承载时（在下一节中描述 HTTP 协议），就称为 Web 服务（Web service）。特别是，简单对象访问协议（SOAP，Simple Object Access Protocol）是一个实现了 Web 服务的方法，它以一种语言无关并且系统无关的方式执行程序之间的 RPC。客户端构造一个 XML 消息的请求，并使用 HTTP 协议把它发送到服务器；服务器返回一个 XML 格式的消息作为应答。这样，异构平台上的应用程序可以进行通信。

回到 AJAX，我们的观点很简单，当运行在浏览器和服务器上的程序需要进行数据交换时，XML 是一种有用的格式。然而，为了在发送或接收数据时为浏览器提供可响应的接

口，它必须尽可能用脚本来执行异步 I/O 而不会阻塞页面的显示，同时等待对一个请求的响应。例如，考虑在浏览器中滚动一幅地图。当浏览器收到滚动动作的通知，地图页面上的脚本可能会向服务器请求更多的地图数据，尤其是显示的地图接近数据的边缘。在获取这些数据时界面不应该呆滞，因为这样的接口不会赢得任何用户的赞赏。相反，地图的滚动应该继续平滑地进行。当数据到达后要通知该脚本以便它可以使使用新到的数据。如果一切顺利，新的地图数据应该在需要它之前获得。现代浏览器都支持这种通信模型。

拼图中的最后一块是将 AJAX 保持住的脚本语言，对上述列表中的技术提供访问方式。在大多数情况下，这种语言是 JavaScript，但也有诸如 VBScript 的替代品。我们之前介绍了 JavaScript 的一个简单例子。注意不要被这种简单性所愚弄，JavaScript 有许多怪癖；但它是一个完全成熟的编程语言，拥有所有 C 或 Java 的能力。它有变量、字符串、数组、对象、函数和所有通常的控制结构。它还具有与浏览器和 Web 页面的特殊接口。JavaScript 可以在屏幕上跟踪鼠标在对象上的移动，这样很容易突然弹出一个菜单，使得 Web 页面看起来更加生动活泼。它还可以使用 DOM 来访问页面、操作 HTML 和 XML，并执行异步 HTTP 通信。

在我们离开动态页面这个主题之前，让我们简要地总结到目前为止涉及的技术，并用一张图把它们联系起来。完整的 Web 页面可以用服务器机器上的各种脚本动态生成。这些脚本可以用诸如 PHP、JSP 或 ASP.NET 这样的服务器扩展语言编写，或者由作为单独的 CGI 进程运行，从而可以用任何语言来编写。这些选项如图 7-35 所示。

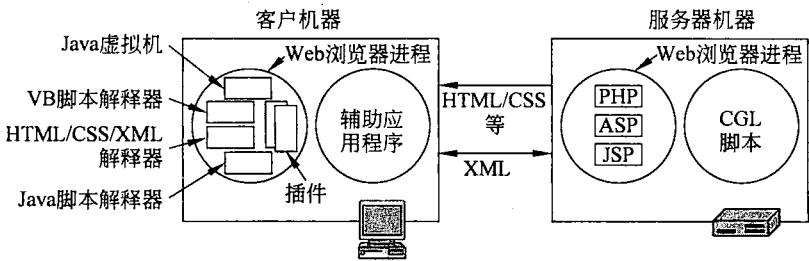


图 7-35 用来生成动态页面的不同技术

一旦浏览器接收到这些网页，就把它们视为 HTML、CSS 和其他 MIME 类型的正常页面对待，并且显示给用户。可以通过安装运行在浏览器内部的插件和运行在浏览器之外的辅助应用程序，来扩展浏览器支持的 MIME 类型。

在客户端生成动态内容也是可能的。嵌入到网页中的程序可以用 JavaScript、VBScript、Java 和其他语言编写。这些程序可以执行任意的计算，并且更新屏幕显示。使用 AJAX，Web 页面中的程序可以异步地与服务器交换 XML 和其他各种数据。这种模式支持丰富的 Web 应用程序，使它们看起来就像传统的应用程序一样，只有一点除外，那就是它们运行在浏览器内而且访问的信息存储在 Internet 的服务器上。

7.3.4 HTTP——超文本传输协议

既然我们已经理解了 Web 内容和应用，现在是时候考察在 Web 服务器和客户之间传输所有这些信息的协议了。这就是超文本传输协议（HTTP，HyperText Transfer Protocol），

由 RFC 2616 说明。

HTTP 是一个简单的请求-响应协议，它通常运行在 TCP 之上。它指定了客户端可能发送给服务器什么样的消息以及得到什么样的响应。请求和响应消息的头以 ASCII 码形式给出，就像 SMTP 一样；而消息内容则具有一个类似 MIME 的格式，也像 SMTP 一样。这个简单模型是早期 Web 成功的有功之臣，因为它使得开发和部署是那么的直截了当。

在本节，我们将着眼于时下使用 HTTP 的一些更加重要的属性。然而，在进入 HTTP 的细节之前，我们注意到它在 Internet 上的使用方式在不断地演化。HTTP 是一个应用层协议，因为它运行在 TCP 之上，并且与 Web 密切相关。这就是为什么我们在本章讨论它。然而，在另一种意义上，HTTP 变得越来越像一个传输层协议，因为它为进程之间跨越不同网络进行内容通信提供了一种方式。这些进程不一定必须是 Web 浏览器和 Web 服务器。媒体播放器也可以使用 HTTP 与服务器通信并请求专辑信息。防病毒软件可以使用 HTTP 来下载最新的病毒库更新。开发人员可以使用 HTTP 来获取项目文件。消费电子产品比如数码相框通常使用内嵌的 HTTP 服务器作为与外界的接口。机器与机器之间的通信越来越多地通过 HTTP 运行。例如，航空公司服务器可以使用 SOAP（一种运行在 HTTP 之上的 XML RPC）联系一家汽车租赁服务器，并且进行汽车预订，所有这些都作为度假产品的一部分。这种趋势有可能继续下去，伴随着 HTTP 的使用而不断扩大。

连接

浏览器与服务器联系最常用的方法是与服务器机器上的端口 80 建立一个 TCP 连接，虽然这个过程并不是正式要求的。使用 TCP 的意义在于浏览器和服务器都不需要担心如何处理长消息、可靠性或拥塞控制。所有这些事情都由 TCP 实现负责处理。

在 Web 早期 HTTP 1.0 中，连接被建立起来以后浏览器只发送一个请求，之后一个响应消息被发回来，再然后 TCP 连接就被释放了。那时，整个 Web 页面通常只包含 HTML 文本，因此这种方法足够用了。很快，Web 页面发展成还含有大量的嵌入式内容链接，比如图标以及其他精美的东西。建立一个单独的 TCP 连接来传输每个图标的操作方式代价太昂贵。

这种现象导致了 HTTP 1.1 的诞生，它支持持续连接（persistent connection）。有了这种连接，就有可能建立一个 TCP 连接，在其上发送一个请求并得到一个响应，然后再发送额外的请求并得到额外的响应。这种策略也称为连接重用（connection reuse）。通过把 TCP 连接的建立、启动和释放等开销分摊到多个请求上，相对于每个请求的 TCP 开销就被大大地降低了。而且，还可以发送流水线请求，也就是说在请求 1 的响应回来之前发送请求 2。

这 3 种情况的性能差异如图 7-36 所示。图 7-36(a)部分显示了 3 个请求，一个接着一个，每一个都是一个单独的连接。让我们假设这表示一个 Web 页面，其中有两个嵌入式图像在同一台服务器上。图像的 URL 被确定为主页，并且已经获取，因此在主页之后这些图像也已经获取过来。如今，一个典型的页面大约有 40 个对象，这些对象必须都取来才能表示出来，但是这将使我们的图太大，所以我们只使用两个嵌入式对象。

在图 7-36(b)中，通过一个持续连接来获取页面。也就是说，TCP 连接在启动后是开放的，然后发送相同的 3 个请求，一个接着一个，后一个跟前一个一样，然后才关闭连接。可以观察到页面的提取任务更快速地完成了。加速的原因有两个：首先，时间没有浪费在

建立更多的连接上。每个 TCP 连接至少需要一个往返时间才能建立；其次，相同图像的传输更加迅速。为什么会这样？这是因为 TCP 的拥塞控制。在一个连接的开始阶段，TCP 使用慢启动过程来增加吞吐量，直到它了解网络路径的行为。这个预热期的后果是多个短 TCP 连接比一个长 TCP 连接传输信息所需的时间更长。

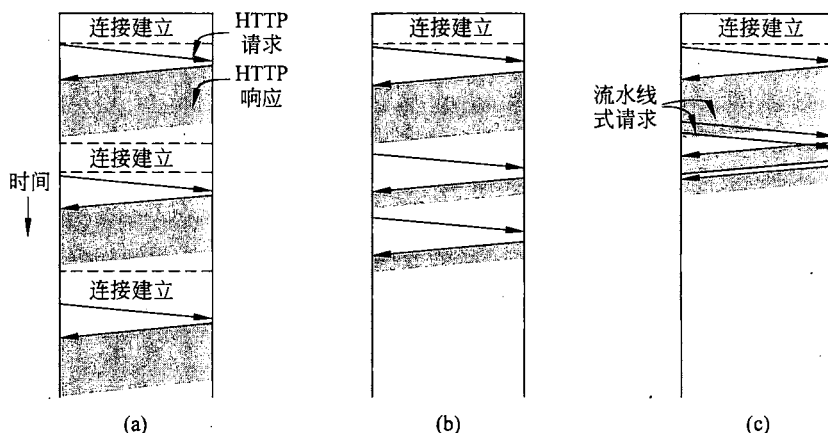


图 7-36 3 种情况下的 HTTP

(a) 具有多个连接和系列请求；(b) 具有一个持续连接和系列请求；(c) 具有一个持续连接和流水线式请求

最后，在图 7-36(c)中，有一个持久连接和请求流水线。具体来说，第二个和第三个要求尽快发送出去，一旦检索主页足够识别出必须预取图像就立即发送请求。这些请求的响应最终紧随其后。这种方法减少了服务器处于空闲状态的时间，因此进一步提高了性能。

然而，持续连接不是免费的。一个新的问题出现了，那就是什么时候关闭连接。一个到服务器的连接在页面加载时应该保持开放。然后呢？对于用户来说有个很好的机会，他可以点击一个链接从服务器请求另一个页面。如果连接保持打开状态，那么下一个请求被立即发送出去。不过，谁也不能保证任意时间后客户将向服务器发出另一个请求。实际上，客户机和服务器通常将持续连接保持打开状态，直到它们已经闲置一小段时间（例如 60 秒），或者它们已经打开了大量的连接必须要关闭一些。

细心的读者可能已经注意到还有一种组合，我们到目前为止还没有提及。那就是，在每个 TCP 连接上发送一个请求，但同时并行打开多个 TCP 连接。这种并行连接（parallel connection）方法在持续连接之前被浏览器广泛使用。它具有与顺序连接相同的缺点——额外的开销，但性能比顺序连接的要好。这是因为建立和加大并行连接隐藏着一些延迟。在我们的例子中，两个嵌入式图像的连接可以在同一时间建立。然而，在同一台服务器上运行多个 TCP 连接会令人沮丧。原因在于 TCP 为每个连接单独执行拥塞控制。因此，连接之间彼此竞争，造成丢包率上升，而且聚合起来比单个连接的网络用户更具侵略性。持续连接则更优越，应该比并行连接优先使用，因为它们避免了开销，而且不会遭受流量拥塞问题。

方法

尽管 HTTP 是为 Web 而设计的，但出于放眼未来面向对象的使用，它的设计有意识地比 Web 所需要的更加通用。正是这个原因，HTTP 不仅支持请求一个 Web 页面，而且支持

操作——称为方法 (method)。这种通用性是 SOAP 得以存在的主要原因。

每个请求由一行或多行 ASCII 文本组成,其中第一行的第一个词是被请求的方法名字。图 7-37 列出了内置的方法。方法名区分大小写,因此 GET 是合法的方法而 get 不是。

方法	描述
GET	读取一个Web页面
HEAD	读取一个Web页面的头
POST	附加一个Web页面
PUT	存储一个Web页面
DELETE	删除一个Web页面
TRACE	回应入境请求
CONNECT	通过代理连接
OPTIONS	一个页面的查询选项

图 7-37 内置的 HTTP 请求方法

GET 方法请求服务器发送页面 (在大多数情况下,当我们说“页面”时我们指的是“对象”,但是把一个页面想象成一个文件,理解这个概念就足够了)。该页面被适当编码成 MIME。大部分发送给 Web 服务器的请求都是 GET 方法。GET 的通用形式是:

```
GET filename HTTP/1.1
```

其中 filename 是预取的页面名字,1.1 是协议版本号。

HEAD 方法只请求消息头,不需要真正的页面。这个方法可以收集建索引所需要的信息,或者只是测试一下 URL 的有效性。

当提交表单时需要用到 POST 方法。POST 方法与 GET 方法都可被用作 SOAP Web 服务。与 GET 类似,它也携带一个 URL,但不是简单地检索一个页面,而是上载数据到服务器 (即表单的内容或者 RPC 参数)。然后,服务器利用这些数据做某件事,具体取决于 URL,概念上是将数据“附加”到对象上。效果或许是购买一个表项,或者调用一个过程。最后,方法返回一个指出结果的页面。

其余的方法对于浏览 Web 并不常用。PUT 方法与 GET 方法相反:它不是读取页面,而是写入页面。通过这个方法可以在远程服务器上建立起一组 Web 页面。这个请求的主体包含了页面。页面可以利用 MIME 来编码,在这种情况下,跟在 PUT 后面的行可能包含了认证头,以便证明调用者的确有权执行所请求的操作。

DELETE 方法的用途可能与你想象的一样:删除页面,或者至少指出 Web 服务器已经同意删除该页面。与 PUT 方法类似,认证和许可机制在这里起到了很重要的作用。

TRACE 方法用于调试。它指示服务器发回收到的请求。当请求没有被正确地处理而客户希望知道服务器实际得到的是什么样的请求时,这个方法非常有用。

CONNECT 方法使得用户通过一个中间设备 (比如 Web 缓存) 与 Web 服务器建立一个连接。

OPTIONS 方法提供了一种办法让客户向服务器查询一个页面并且获得可用于该页面的方法和头。

每个请求都会得到一个响应,每个响应消息由一个状态行及可能的附加信息 (例如全部或者部分 Web 页面) 组成。状态行包括一个 3 位数字的状态码,该状态码指明了这个请求是否被满足;如果没有满足,那么原因是什么。第一个数字把响应分成 5 大组,如图 7-38

所示。1××码实际上很少被使用。2××码意味着这个请求被成功地处理，并且返回了相应的内容（如果有的话）。3××码告诉客户应该检查其他地方：使用另一个不同的 URL，或者在它自己的缓存中查找（后面讨论）。4××码意味着由于客户错误而导致请求失败，比如无效请求或者不存在的页面。最后，5××错误码意味着服务器自身出现内部问题，有可能是服务器代码中有错误，也可能是临时负载过重。

代码	含义	例子
1××	信息	100 = 服务器同意处理客户请求
2××	成功	200 = 请求成功；204 = 没有内容
3××	重定向	301 = 移动页面；304 = 缓存的页面仍然有效
4××	客户错误	403 = 禁止页面；404 = 页面没找到
5××	服务器错误	500 = 服务器内部错误；503 = 稍后再试

图 7-38 响应组的状态码

消息头

请求行（例如 GET 方法的行）后面可能还有额外的行，其中包含了更多的信息。它们同称为请求头（request header）。这些信息可以与一个过程调用的参数相类比。响应消息也有响应头（response header）。有些头可以用在两个方向上。图 7-39 列出的是一些最重要的

头	类型	内容
User-Agent	请求	有关浏览器及其平台的信息
Accept	请求	客户可处理的页面类型
Accept-Charset	请求	客户可接受的字符集
Accept-Encoding	请求	客户可处理的页面编码
Accept-Language	请求	客户可处理的自然语言
If-Modified-Since	请求	检查新鲜度的时间和日期
If-None-Match	请求	先前为检查新鲜度而发送的标签
Host	请求	服务器的 DNS 名字
Authorization	请求	列出客户的信任凭据
Referer	请求	发出请求的先前 URL
Cookie	请求	给服务器发回 Cookie 的先前 URL
Set-Cookie	响应	客户存储的 Cookie
Server	响应	关于服务器的信息
Content-Encoding	响应	内容如何编码（比如，gzip）
Content-Language	响应	页面使用的自然语言
Content-Length	响应	页面以字节计的长度
Content-Type	响应	页面的 MIME 类型
Content-Range	响应	标识了页面内容的一部分
Last-Modified	响应	页面最后修改的时间和日期
Expires	响应	页面不再有效的时间和日期
Location	响应	告诉客户向谁发送请求
Accept-Ranges	响应	指出服务器能接受的请求的字节范围
Date	请求/响应	发送消息的日期和时间
Range	请求/响应	标识一个页面的一部分
Cache-Control	请求/响应	指示如何处理缓存
ETag	请求/响应	页面内容的标签
Upgrade	请求/响应	发送方希望切换的协议

图 7-39 某些 HTTP 消息头

消息头。这个列表不短，正如你可以想象的那样，每个请求和响应通常具有不同的头。

User-Agent 头允许客户将它的浏览器实现告知服务器（比如，Mozilla/5.0 和 Chrome/5.0.375.125）。这些信息非常有用，服务器可以据此裁剪给浏览器的响应，因为不同浏览器的能力和行为大相径庭。

如果客户对于可接受的信息有一定的限制，可以用 4 个 **Accept** 头告诉服务器愿意接受什么。第一个头指定了哪些 MIME 类型是可以接受的（例如，text/html）。第二个头给出了字符集（比如 ISO-8859-5 或者 Unicode-1-1）。第三个头处理压缩方法（比如 gzip）。第四个头指明了一种自然语言（例如西班牙语）。如果服务器有多个页面可供选择的话，那么它可以利用这些信息向客户提供所需要的页面。如果客户的请求不能满足，则返回一个错误码并且请求失败。

If-Modified-Since 和 **If-None-Match** 头用于缓存。它们允许客户在网页的缓存副本不再有效时请求该网页。我们稍后将描述缓存机制。

Host 头是服务器的名字，它取自 URL。这个头是强制性质的。有些 IP 地址可能对应了多个 DNS 名字，所以服务器需要某种方法来分辨出应该把请求传递给哪个主机来处理。

对于那些被保护的页面，**Authorization** 头是必需的。在这种情况下，客户可能需要证明自己有权利来查看被请求的页面。这个头就是用在这种情况下。

客户端使用拼写错误的 **Referer** 头给出了请求当前页面的 URL。大多数情况下，这是前一页的 URL。这个头对于跟踪 Web 浏览特别有用，因为它告诉了服务器客户如何到达该页面的。

尽管 **Cookie** 是在 RFC 2109 中而不是 RFC 2616 中被阐述的，它们也有头。**Set-Cookie** 头是服务器向客户端发送 **Cookie**，期待客户端保存 **Cookie**，并且在后续的请求中返回给服务器，返回时要使用 **Cookie** 头（请注意，针对 **Cookie** 最近出了个规范说明了一种新的头，即 RFC 2965。但是它在很大程度上已经遭到行业拒绝，不太可能得到广泛实施）。

响应消息中还用到了许多其他的头。**Server** 头允许服务器在愿意时标识构建它的软件。下面 5 个头，都从 **Content-**起始，允许服务器描述它发送的页面的属性。

Last-Modified 头说明了页面最后被修改的时间，**Expires** 头说明了页面能保持有效多长时间。这两个头在页面缓存机制中发挥着重要的作用。

Location 头被服务器用来通知客户，它应该尝试另外一个不同的 URL。如果页面已经被移动，或者允许多个 URL 指向同一个页面（可能在不同的服务器上），则可以使用这个头。它也可被用于另外一种情况，即公司的主页在 com 域中，但是根据客户的 IP 地址或者首选的语言，将客户的请求重定向到一个国家的或者地区的页面中。

如果一个页面非常大，则小客户可能不想一次获取所有的内容。有些服务器可以接受对于有字节范围限定的请求，因此该页面可以通过多个小单位的请求取回来。**Accept-Ranges** 头声明了服务器愿意处理这种部分页面请求。

现在我们来到了可以在两个方向上同时使用的头。**Date** 头可用在两个方向上，并且包含了发送消息的日期和时间，而 **Range** 头则告诉了响应消息提供的网页的字节范围。

ETag 头给出了一个简短的标签，作为页面内容的名称。它主要被用于缓存。**Cache-Control** 头给出了其他有关如何缓存（或更通常的是，如何不缓存）网页的明确指示。

最后，**Upgrade** 头用来切换到一个新的通信协议，比如未来的 HTTP 协议或安全的传

输协议。它允许客户宣布自己可以支持什么协议，同时允许服务器声称自己正在使用什么协议。

缓存

人们往往会返回到以前浏览过的 Web 页面，而且相关的网页往往具有相同的嵌入式资源。比如有些例子包括用于整个网站导航的图像，以及常见的样式表和脚本。如果每次显示这些页面都要去服务器获取全部的资源，这将非常浪费，因为浏览器已经有了一个副本。

积攒已经获取的网页供日后使用的处理方式称为**缓存（caching）**。其优点是当缓存的页面被重复使用时，没有必要进行重复传输。HTTP 内置了一种技术支撑，帮助客户标识他们何时可以放心地重用页面。这种支撑能减少网络流量和延迟，从而提高了性能。但现在存在一种权衡，即浏览器必须存储页面。因为本地存储成本已经很低廉，因此这几乎总是一个值得做的选项。这些网页通常保存在磁盘上，当稍后浏览器运行时可以直接使用它们。

HTTP 缓存的困难问题在于如何确定以前缓存的页面和将要重新获取的页面是相同的。这个确定不能仅依靠 URL 来作出。例如，URL 可能会给出一个页面，显示了最新的新闻项目。即使 URL 保持不变，该页面的内容也会经常更新。另外一种情况是，页面的内容可能是来自希腊和罗马神话中的诸神列表，那么这种页面的改变速度应该不那么快。

HTTP 使用两种策略来解决这个问题。这些策略如图 7-40 所示，作为请求（第 1 步）和响应（第 5 步）之间的处理形式。第一种策略是页面验证（第 2 步）。访问高速缓存，如果对所请求的 URL 它有该页面的副本，而且该副本已知是新鲜的（即仍然有效），那么就没有必要从服务器重新获取。此时，直接返回缓存的页面。该缓存页面最初获取时返回的 Expires 头以及当前的日期和时间可以被用来作出该副本是否有效的决定。

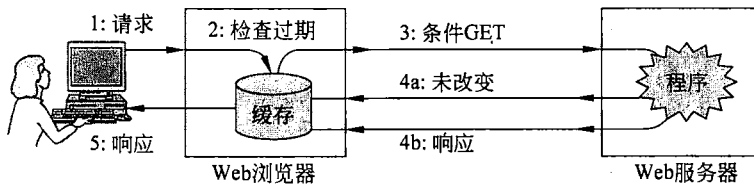


图 7-40 HTTP 缓存

然而，并非所有的网页都有个方便的 Expires 头来告诉你何时必须重新获取网页。毕竟，做出预测才是硬道理——尤其关于未来的预测。在这种情况下，浏览器可能使用启发式的方法来作出决策。例如，如果页面在过去的一年尚未修改（从 Last-Modified 头获知），那么它极有可能在接下来的一个小时里不会有所改变，这或许是一个相当安全的赌注。然而，这里没有任何保证，也许下了个坏的赌注。例如，股市已经关闭一天，因此该页面数个小时都不曾改变，但下一个交易时段开始后它会迅速改变。因此，一个页面的缓存性随着时间的推移可能变化很大。基于这个原因，应谨慎使用启发式策略，尽管它们往往实际工作得挺好。

找到没有过期的页面是使用缓存的最大收益，因为这意味着根本不需要联系服务器。不幸的是，它并不总是能工作得很好。服务器必须谨慎地使用 Expires 头，因为它们可能无法确定一个页面何时将被更新。因此，缓存的副本可能仍然是新鲜的，但客户并不知道。

第二个策略用在这种情况下。它询问服务器缓存的副本是否仍然有效。这个请求是一个条件 GET (conditional GET), 如图 7-40 中第 3 步所示。如果服务器知道缓存的副本仍然是有效的, 它可以发送一个简短的答复说是的 (第 4a 步)。否则, 它必须发送完整的响应消息 (第 4b 步)。

更多的头字段可用来让服务器检查一个缓存的副本是否仍然有效。客户端从 Last-Modified 头可以获知缓存页面的最后更新时间。它可以使用 If-Modified-Since 头将该时间发送给服务器, 询问服务器所请求的页面在此期间是否发生过改变。

另一种选择是, 服务器可能会返回一个包含 ETag 头的页面。这个头给出了一个标签, 是页面内容的一个短名称, 有点像校验和但比它更好。(它可以是一个加密的哈希值, 我们将在第 8 章中描述)。客户端可以向服务器请求验证缓存的副本, 具体做法是给服务器发送 If-None-Match 头, 该头列出了缓存副本的标签。如果任何一个标签与内容相匹配, 服务器将会予以响应, 相应的缓存的副本就可以使用。在不太方便时可以使用这种方法, 这种方法对确定页面是否新鲜很有用。例如, 一个服务器可能会为相同的 URL 返回不同的内容, 这完全取决于首选什么样的语言和 MIME 类型。在这种情况下, 仅仅一个修改日期无助于服务器确定缓存页面是否新鲜。

最后, 请注意这两种缓存策略都会被 Cache-Control 头携带的指令所覆盖。当页面不恰当被缓存时, 这些指令可以被用来限制缓存 (例如, no-cache)。比如, 下一次抓取的动态页面将是不同的, 因此不能缓存; 或者那些需要授权的页面也不能被缓存。

关于如何缓存的方法有很多, 但我们只有空间谈两个要点。首先, 缓存可以设置在除浏览器之外的其他地方。一般情况下, HTTP 请求可以通过一系列的缓存路由。使用浏览器之外的外部高速缓存方法称为代理缓存 (proxy caching)。每增加一个缓存级别可有助于进一步减少请求链。诸如 ISP 和企业这些组织经常使用这种缓存方法, 它们运行代理缓存为不同的用户提供了从缓存页面尽快获得响应的好处。我们将在本章结尾处的 7.5 节关于更广泛的内容分布主题中再来讨论代理缓存。

其次, 高速缓存对提升性能是个很大的推动, 但它并没有人们所希望的那么多。原因是虽然在 Web 上肯定有许多流行的文档, 但也有很多人获取的文档不是那么流行, 其中很多还很长 (例如, 视频)。不受欢迎的“长尾巴”文档占用了缓存空间, 而且缓存能处理的请求数量随着高速缓存大小的增加而增长缓慢。Web 高速缓存能够处理的请求数量可能总是达不到全部请求数量的一半。对于这方面的更多信息, 请参考 (Breslau 等, 1999)。

HTTP 实验

因为 HTTP 是一个 ASCII 协议, 所以对于一个坐在终端 (浏览器的对面) 前面的人来说很容易与 Web 服务器直接通话。这里所需要的全部只是一个连到服务器端口 80 的 TCP 连接。鼓励读者亲自尝试用下面的命令序列来做实验。在大多数 UNIX shell 和 Windows 的命令窗口下这些命令都可以工作 (只要 telnet 程序可用)。

```
telnet www.ietf.org 80
GET /rfc.html HTTP/1.1
Host: www.ietf.org
```

这个命令序列启动一个 telnet 连接 (即 TCP 连接), 该连接指向端口为 80 的 IETF 的

Web 服务器 www.ietf.org。然后是 GET 命令，指定了 URL 的路径和协议。尝试你自己选择的服务器和 URL。接下来一行是强制的 Host 头。最后一个头后面必须跟一个空行。它告诉服务器没有更多的请求头了。然后服务器发送响应消息。根据不同的服务器和 URL，可以观察到不同类型的头和页面。

7.3.5 移动 Web

Web 通常被大多数类型的计算机所用，这其中包括移动电话。在移动的同时通过无线网络浏览网页非常有用，但要做到这点面临着许多技术问题，因为太多的 Web 内容是为具有宽带连接并有华丽表示的台式计算机而设计的。在本节，我们将描述如何从移动设备访问 Web，或者移动 Web（mobile Web），这些技术正处在开发之中。

相比工作或家庭用的台式计算机，用移动电话来浏览 Web 存在几个方面的困难：

- (1) 相对小的屏幕妨碍了大页面和大图像的显示。
- (2) 有限的输入能力使得 URL 的输入很乏味或者输入法很慢。
- (3) 无线链路的网络带宽有限，尤其是蜂窝（3G）网络，费用还很高。
- (4) 网络的连通性断断续续。
- (5) 由于电池寿命、大小、散热以及成本等诸多原因，导致计算能力有限。

这些困难意味着简单地把台式机内容作为移动 Web 很可能会传递出一种令人懊恼的用户体验。

移动 Web 的早期方法采用了一种新的协议栈，这种协议栈专门针对能力有限的无线设备而发明出来的。无线应用协议（WAP，Wireless Application Protocol）是这种策略的最知名的例子。经过各大移动电话厂商的努力，1997 年 WAP 正式启动，其中包括诺基亚、爱立信和摩托罗拉公司。然而，一路上意想不到的事情发生了。在接下来的十年间，随着 3G 数据服务的部署，以及移动电话有了更大的彩色显示屏、更快的处理器和 802.11 无线功能，网络带宽和设备能力得到了巨大的提高。突然间，完全有可能在移动电话上运行简单的 Web 浏览器。虽然这些移动电话与台式机之间始终存在着一个沟壑，并且这个沟壑永远也不会消除，但是当初推动一个单独协议栈的技术问题现在已经淡化。

当前使用得越来越多的方法是移动电话和台式机运行相同的 Web 协议，并且当用户恰好在移动设备上时 Web 站点负责提供移动友好（mobile-friendly）的内容。通过查看请求头信息，Web 服务器能够检测出应该返回桌面版本的网页还是移动版本的网页。User-Agent 头在这方面特别有用，因为它标识了浏览器软件。因此，当 Web 服务器接收到一个请求时，它可能首先查看头，然后给 iPhone 返回图像小、文字少和简单的导航页面，而给笔记本电脑用户返回一个全功能的网页。

W3C 从几个方面鼓励这种移动终端和计算机通用的方法。一种方式是标准化移动 Web 内容的最佳实践（best practice）。第一个规范提供了 60 个这样的最佳实践（Rabin 和 McCathieNevile，2008）。因为通信成本高于计算成本，其中的大多数实践采取了明智的措施来减少页面的大小，包括使用压缩算法，并且最大限度地提高缓存的有效性。这种方法鼓励了网站，尤其是大型网站创建移动 Web 版本的页面内容，因为这是虏获移动 Web 用户所必须要做的事情。为了帮助这些用户，还有一个标志指示可以在移动 Web 上（很好）

浏览的页面。

另一种有用的工具是一个精简版的 HTML，称为**基本 XHTML** (XHTML Basic)。这种语言是 XHTML 的一个子集，专用于移动电话、电视机、个人数字助理、自动售货机、传呼机、汽车、游戏机，甚至手表。出于这个原因，它不支持样式表、脚本或框，但大多数的标准标签它都有。它们被分成 11 个模块。有些是必需的，而有些是可选的。所有的模块都以 XML 定义。图 7-41 列出了一些模块和标签例子。

模块	需要否	功能	标签实例
Structure	是	文档结构	body, head, html, title
Text	是	信息	br, code, dfn, em, hn, kbd, p, strong
Hypertext	是	超链接	a
List	是	明细表	dl, dt, dd, ol, ul, li
Forms	否	填入表单	form, input, label, option, textarea
Tables	否	矩形表格	caption, table, td, th, tr
Image	否	图像	img
Object	否	Java小程序、地图等	object, param
Meta-information	否	额外的信息	meta
Link	否	类似于<a>	link
Base	否	URL起始点	base

图 7-41 基本 XHTML 模块和标签

然而，并非所有的页面都能设计成在移动 Web 上工作良好。因此，一种互补的方法是使用**内容转换或转码** (content transformation or transcoding)。在这种方法中，将一台计算机设置在移动电话和服务器之间，它从移动电话获得请求，然后从服务器预取内容，最后把它转换成移动 Web 内容。一种非常简单的转换方法是减少大型图片的尺寸，将它重新格式化成一个较低的分辨率。当然还可以使用其他许多小而有用的转换方法。自从移动 Web 出现以来，转码技术的使用已经取得了一些成功。例如，有关这方面的信息可参考 (Fox 等, 1996)。然而，当这两种方法都使用时，究竟由服务器还是转码器来作出移动内容的决策，两者之间存在着一种紧张关系。例如，一个 Web 站点针对移动 Web 可能会选择一种图像和文字的特殊组合，只能由一个转码器来改变图像的格式。

我们讨论至今一直关注着 Web 内容，而没有涉及协议，因为实现移动 Web 的最大问题正是由内容引起的。然而，我们简要提及协议问题。Web 所用的 HTTP、TCP 和 IP 协议可能消耗掉相当数量的带宽，因为这些协议的开销不小 (比如协议头)。为了解决这个问题，WAP 和其他解决方案定义了特殊用途的协议。这在很大程度上是没有必要的。报头压缩技术，比如第 6 章中表述的鲁棒报头压缩 (ROHC, RObust Header Compression) 可以减少这些协议的开销。通过这种方式，很可能只需要一组协议 (HTTP, TCP, IP)，无论是高带宽还是低带宽链路都使用这组协议。在低带宽链路上使用时只需要打开头压缩技术。

7.3.6 Web 搜索

为了完成对 Web 的描述，我们将讨论可以说是最成功的 Web 应用程序：搜索。1998 年，时任斯坦福大学研究生的 Sergey Brin 和 Larry Page 组建了一个称为谷歌 (Google) 的新兴公司，目标是建立一个更好的网络搜索引擎。他们秉持当时激进的想法：一个搜索算

法计算每个页面多少次被其他页面所指是评价该页面重要性的更好手段，比每个页面包含多少个被搜索关键词的手段更好。例如，许多页面链接到思科（Cisco）的主页，对一个搜索“Cisco”的用户而言，这个主页比思科以外的公司碰巧有个页面多次出现“思科”的页面要重要得多。

他们是对的。已经证明可以建立一个更好的搜索引擎，于是人们蜂拥而至。有了风险资本的支持，谷歌大幅增长。它在 2004 年成为一家上市公司，市值 230 亿美元。到 2010 年，据估计它分布在世界各地的数据中心有超过 100 万台服务器在运行。

从某种意义上说，搜索仅仅是另一个 Web 应用程序，尽管它是最成熟的 Web 应用程序之一，因为自 Web 初期以来它一直在被不断地开发。然而，Web 搜索已经成为日常生活中不可缺少的应用。估计每天有超过 10 亿个网页被搜索。寻找所有其他信息方式的人把搜索作为起始点。例如，要找出在西雅图哪里有卖蔬菜和调味品，刚开始没有明显的网站可以使用。但机会存在于搜索引擎知道一个网页具有所需的信息，因而可以快速引导你找到答案。

要以传统的方式进行 Web 搜索，用户将她的浏览器指向一个 Web 搜索网站的 URL。主要的搜索网站包括谷歌、雅虎和 Bing。接下来，用户使用表单提交搜索词。这种行为导致搜索引擎对它的数据库执行一次查询，搜索相关的页面或图像，或者想搜索的任何类型的资源，并且将查询结果作为一个动态页面返回。然后，用户可以按照链接找到已被发现的网页。

Web 搜索是进行讨论的一个有趣话题，因为它对网络的设计和使用有很大影响。首先，存在一个 Web 搜索如何找到网页的问题。Web 搜索引擎必须有一个运行查询算法的页面数据库。每个 HTML 页面可能包含指向其他页面的链接，而且每一件有趣的事情（或者至少是可搜索的）链接在某个地方。这意味着，从少数页面开始通过遍历所有网页和链接，在理论上可能找出 Web 上的所有其他页面。这个过程称为 **Web 爬虫**（Web crawling）。所有的 Web 搜索引擎均使用 **Web 爬虫程序**（Web crawler）。

与爬虫有关的一个问题是它能找到什么样的页面。获取静态文件，然后跟随链接很容易做到。然而，许多网页包含一些程序，根据用户的交互显示不同页面。其中一个例子是在线商店的目录。该目录可能包含根据产品数据库创建的动态页面，以及针对不同产品的查询。这种类型的内容不同于很容易遍历的静态页面。Web 爬虫如何找到这些动态页面？答案是，在大多数情况下，它们抓取不到。这种隐藏的内容称为 **深层 Web**（deep Web）。如何搜索深部 Web 是一个开放的问题，研究人员正在努力解决之。例如，有关这方面的信息可参考（Madhavan 等，2008）。也有一些约定，网站提供一个页面（称为 robots.txt）告诉爬虫程序该站点的哪些部分应该或者不应该访问。

第二个需要考虑的问题是如何处理所有爬虫抓到的数据。为了使得索引算法能在大量的数据上运行，页面必须被有效存储起来。虽然估计值在不断地变化，但主要的搜索引擎被认为具有数百亿页面的索引，这些页面取自 Web 的可见部分。平均页面大小大约为 320 KB。这些数字意味着，抓取一个 Web 的副本需要 20 PB 量级或 2×10^{16} 个字节量级的存储量。虽然这是一个真正庞大的数字，但它是 Internet 数据中心可以轻松存储和处理的数据量（Chang 等，2006）。例如，如果磁盘的存储成本 20 美元/TB，那么 2×10^4 TB 的成本为 400 000 美元，这个数字对于像谷歌、微软和雅虎这样的公司实在不能算大。虽然网

络还在扩大,但磁盘的成本也在急剧下降,因此存储整个 Web 在可预见的未来对于大公司仍然是可行的。

要使这些数据有意义则是另外一回事。你或许欣赏 XML 如何帮助程序轻松地提取数据结构,自主特设的格式会导致很多猜测。还有格式之间的转换问题,甚至是语言之间的翻译。但是,即使知道数据的结构也只是解决了问题的一部分。明白它的意思才是硬道理。从更相关的结果页面开始搜索查询,这是可以获得更大价值的关键所在。最终目标是要能够回答问题,例如,在你城市的哪里能买到便宜但像样的烤箱。

Web 搜索的第三个方面是它已经提供了更高层次的命名。没有必要记住一个长长的 URL,假设你记名字比记 URL 记得更好,那么用一个人的名字来搜索网页一样可靠(或者更多)。这一策略越来越成功。犹如 DNS 域名贬低了计算机的 IP 地址,Web 搜索降低了计算机的 URL。此外,有利于搜索的是它能纠正拼写和输入错误,否则如果你输入了一个错误的 URL,你只能得到错误的页面。

最后,Web 搜索向我们显示的事情很少与网络设计有关,但与某些 Internet 服务增长有关的是:广告中有太多的钱。广告是驱动 Web 搜索增长的经济引擎。从印刷广告到 Web 广告的主要变化在于:Web 广告具备根据人们正在寻找的对象来发放广告的能力,以此提高了广告的相关性。拍卖机制的变异方法被用来匹配搜索查询与最有价值的广告(Edelman 等,2007)。当然,这种新的模式已经引起了新的问题,比如“点击欺诈”,在这种情况下,程序模仿用户点击广告造成没有真正赢利的报酬。

7.4 流式音视频

Web 应用和移动 Web 并不是网络应用领域中唯一令人振奋的发展。对许多人来说,音频和视频才是网络的圣杯。当提及“多媒体”这个词时,无论技术人员和商人都会不约而同地垂涎三尺。前者看到的是为每一台计算机提供 IP 语音和视频点播所隐含的巨大技术挑战,而后者则看到它同样隐含的丰厚的利润。

虽然至少从 20 世纪 70 年代开始就有了通过 Internet 发送音频和视频的想法,但大约自 2000 开始实时音频(real-time audio)和实时视频(real-time video)流量才有了真正极度的增长。实时流量与 Web 流量有很大的不同,它必须以预先设定的速率播放才有用。毕竟,观看一个忽停忽动的慢动作视频不是大多数人的乐趣。相反,Web 可以有短暂的中断,而且页面加载可以花费更多或更少的时间,在一定限度内这不是一个主要的问题。

发生的两件事情极大地促进了这种实时流量的增长。首先,计算机变得更加强大,并且都配备了麦克风和摄像机,因此它们很容易地就能够输入、处理和输出音频及视频数据。其次,Internet 带宽的大量扩张已经大大提高了 Internet 的可用性。Internet 核心骨干网的长途链路具有许多个 kbps,而且宽带和 802.11 无线已经达到 Internet 边缘用户。这些发展使 ISP 可以在它们的骨干网携带巨大的流量,而且这意味着普通用户可以用比 56 kbps 的电话调制解调器快上 100~1000 倍的速度连接到 Internet。

带宽扩张导致音频和视频流量的增长,但这里也有不同的原因。电话呼叫占用相对较少的带宽(原则上是 64 kbps,压缩后更少),但电话服务费历来比较昂贵。公司从中看到

了契机，如果使用现有的带宽通过 Internet 承载语音流量，那么就可以减少他们的电话账单。比如 Skype 这样的新兴公司找到了一个方法，让客户使用他们的 Internet 连接拨打免费电话。傲慢自负的电话公司也找到了一种廉价方式，使用 IP 网络设备进行传统的语音通话。其结果是通过 Internet 网络承载语音数据的服务发生了大爆炸，这种方式称为 IP 语音（voice over IP）或 Internet 电话（Internet telephony）。

与音频不同，视频占用了大量的带宽。合理质量的 Internet 视频编码压缩率大约为 1 Mbps，一个典型 DVD 电影有 2 GB 的数据。在宽带 Internet 接入前，通过网络发送电影简直令人望而却步。现在已今非昔比了。随着宽带的传播，用户在家观看令人满意的视频流终于成为可能，对此人们乐此不疲。对任何一天的估计表明大约四分之一的 Internet 用户会访问 YouTube，这是一个广受欢迎的视频共享网站。电影租赁业务已经转移到在线下载。而且规模庞大的视频改变了 Internet 流量的整体构成。大多数 Internet 流量早已经是视频，据估计在几年内 90% 的 Internet 流量将是视频（Cisco, 2010）。

由于有足够的带宽来承载音频和视频，那么设计音视频流和视频会议应用的关键问题就是网络的延迟。音频和视频需要实时演示，这意味着它们必须以预定速率播放出来才有用。长延迟意味着应该互动的呼叫不再能互动得起来。这个问题是很清楚的，如果你曾经用过卫星电话就应该有所体会，当延迟达到半秒钟时相当的令人分散注意力。在网络上播放音乐或电影，绝对的延迟并不要紧，因为它仅仅影响到媒体何时开始播放。但延迟的变化——称为抖动（jitter），仍然非常重要。抖动必须被播放器掩盖，否则播放出来的音频听起来令人难以理解，播放出来的视频则会看上去很生涩或呆滞。

在本节中，我们将讨论如何处理延迟问题的某些策略，以及用来设置音频和视频会话的协议。在介绍完数字音频和视频后，我们的介绍将分成三种情况，分别采用了不同的设计。第一，也是最简单的情况是如何处理存储的流媒体，就像观看 YouTube 上的视频一样。接下来的一种情况难度适中，就是如何处理流媒体直播。流媒体直播有两个例子，分别是 Internet 广播和 IPTV，其中广播电台和电视台通过 Internet 给许多用户进行直播。最后，也是最困难的情况是如何处理 Skype，或者更普遍交互式音频和视频会议呼叫。

顺便说一句，多媒体（multimedia）这个术语经常出现在 Internet 的上下文中，它意味着视频和音频。从字面上看，多媒体是两个或两个以上的媒体。按照这个定义，这本书就是一个多媒体范例，因为它包含了文字和图形（数字）。然而，这可能不是你心目中的多媒体，因此我们使用“多媒体”一词意味着两个或两个以上的连续媒体（continuous media），也就是说，媒体必须以某种良好定义的时间间隔播放。两个媒体通常指有音频的视频，也就是说，有声音的移动图像。许多人把单纯的音频称为多媒体，比如 Internet 电话或 Internet 广播，这显然不是很恰当。其实，所有这些情况下应该使用的一个更好术语是流媒体（streaming media）。尽管如此，我们跟随大流，把实时音频也当作多媒体来考虑。

7.4.1 数字音频

音频（声音）波是一种一维声（压力）波。当声波进入耳朵时，鼓膜发生振动，从而引起内耳的微细感骨与之一同振动，并向大脑发送神经脉冲。听者感觉到的这些脉冲就是声音。类似的方式是当声波碰击麦克风时，麦克风产生一个电信号，该电信号将声音振幅

表示为时间的函数。

人耳能听到的声音频率范围是 20~20 000 Hz。有些动物，特别是狗，能够听到更高的频率。耳朵听见的声音符合对数关系，因此，功率为 A 和 B 的两种声音的比率，通常被表示成分贝 (dB, decibels): $\text{dB}=10\log_{10}(A/B)$ 。如果我们将 1 kHz 正弦波的能听度低限 (约 20 微帕的声音压力) 定义为 0 dB，那么普通谈话大约是 50 dB，耳朵感觉到痛的阈值大约是 120 dB。可见变化的范围达 100 万倍。

令人惊讶的是人耳对于持续时间仅仅几毫秒的声音变化也异常的敏感。相反地，眼睛却察觉不到持续时间为几毫秒的光度变化。这种观察的结果是多媒体播放中仅仅几毫秒的抖动对于可感觉到的声音质量的影响，要远远大于对可感觉到的图像质量的影响。

数字音频是声波的数字表示，这种表示能重现音频。通过模数转换器 (ADC, Analog Digital Converter) 可以将音频波转换成数字形式。ADC 以电压作为输入，生成一个二进制数作为输出。在图 7-42(a)中，我们看到一个正弦波的例子。为了用数字方式来表示该信号，我们可以每隔 ΔT 秒对它采样一次，如图 7-42(b)中的竖条状所示。如果一个声波不是纯正弦波，而是若干正弦波的线性叠加，并且其中最高的频率成分为 f ，那么，根据尼奎斯特定理 (参见第 2 章)，以 $2f$ 的频率对该信号采样足够了。更高的采样频率没有意义，因为更高采样获得的更高频率这里无法检测出来。

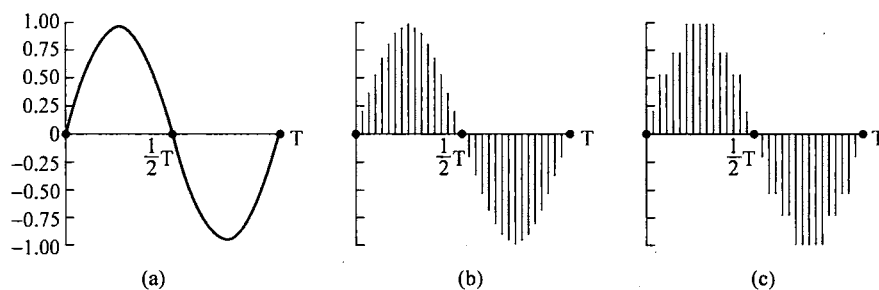


图 7-42

(a) 一个正弦波 (b) 正弦波样值 (c) 量化成 4 位

相反的过程是获取数字值并且产生一个模拟电压。这个过程由数模转换器 (DAC, Digital-to-Analog Converter) 来完成。然后，扬声器将模拟电压转换成声波发送出去，因此人们就能听到声音了。

数字采样永远不可能做到精确。图 7-42(c)中的样值只允许 9 个值，从 -1.00~+1.00，步长为 0.25。一个 8 位的采样允许有 256 个不同的值。一个 16 位的采样可以有 65 536 个不同的值。由于每个采样值的位数有限而引入的误差称为量化噪声 (quantization noise)。如果量化噪声太大，人耳就能觉察得出来。

使用声音采样技术的两个众所周知例子是电话和音频压缩光盘 (CD, compact disc)。电话系统使用的脉冲编码调制，以每秒 8000 次的频率采样，并用 8 位表示采样值。为了最小化感觉到的失真，量化尺度是非线性的；而且限定在每秒只有 8000 采样，任何高于 4 kHz 的频率都将被丢弃。在北美和日本，采用了 μ -l 规则编码技术；在欧洲和国际上则广泛采用 A-l 规则编码技术。每种编码技术的数据率都是 64 000 bps。

音频 CD 是数字化的，其采样率为每秒 44 100 次，这足以捕获高达 22 050 Hz 的频率；

22 050 Hz 对于人来说已经非常好了，但对于犬类音乐爱好者而言却很糟糕。每个样值都用 16 位表示，并且在振幅范围内量化尺度是线性的。但是请注意，16 位样值只有 65 536 个不同的值，甚至人耳可识别的动态声音范围大约为 100 万。因此，纵使 CD 质量的音频远远好于电话质量的音频，但如果每个样值仅使用 16 位，因而引入了一定的量化噪声（尽管 CD 并没有覆盖整个动态范围——CD 不应该会刺伤或刺痛人耳）。一些狂热的音乐发烧友还是喜欢 33-RPM 的 LP 唱片比 CD 更多一些，因为唱片没有截取 22 kHz 的尼奎斯特频率，而且没有量化噪声（但必须非常小心放置，否则会有划痕）。在每秒 44 100 个采样并且每个样值 16 位的情况下，无压缩的 CD 质量音频单声道需要 705.6 kbps 的带宽，立体声则需要 1.411 Mbps 的带宽。

音频压缩

为了减少带宽需求和传输时间，音频往往被进行了压缩处理，即使音频数据率比视频数据传输速率要低得多。所有的压缩系统需要两种算法：一种用在源端上压缩数据，另一种用在接收方上解压数据。在文献中，这些算法分别称为编码（encoding）算法和解码（decoding）算法。我们将沿用这个术语。

压缩算法表现出一定的不对称性，理解这点非常重要。我们首先考虑音频，这种不对称性对于视频也是适用的。对于许多应用程序来说，多媒体文件只被编码一次（当它存储在多媒体服务器上时），但将被解码几千次（每当被客户播放时）。这种不对称性意味着，编码算法较慢而且需要昂贵的硬件是可以接受的，只要解码算法速度快而且不需要昂贵的硬件。一个流行音频（或视频）服务器的运营商可能会很乐意买一个计算机集群，来为它的整个数字图书馆编码，但如果要求客户做同样的事情才能听到音乐或看到电影，那么该运营商恐怕不会成功。许多实用的压缩系统竭尽全力使得解码算法既快速又简单，甚至以编码算法缓慢而且复杂性高作为代价。

另一方面，对于直播的音频和视频，比如 IP 语音呼叫，缓慢的编码是不可接受的。编码必须实时地动态执行。因此，相比存储在磁盘上的音频或视频，实时多媒体使用不同的算法或参数，往往适当减少压缩。

第二个不对称性在于编码/解码过程不必是可逆的。也就是说，压缩数据文件、传输、然后解压缩它，用户期望得到原始的准确的数据，直到最后一位数据。对于多媒体而言，这一要求并不一定存在。音频（或视频）信号经过编码，然后再被解码，得到的信号与原始的略有不同，这通常是可以接受的，只要它听起来（或看起来）和原始的一样即可。当解码输出的信号不完全等同于原始输入的信号，就说该系统是有损耗的（lossy）。如果输入和输出完全相同，则该系统就是无损的（lossless）。有损系统很重要，因为接受少量信息的丢失，通常意味着在可能的压缩率方面能获得巨大的回报。

从历史上看，电话网络中的长途带宽非常昂贵，所以工作中需要一个坚实的声码器（是“语音编码器”的简称），声码器（vocoder）负责为特殊语音进行压缩音频。人的讲话往往在 600~6000 Hz 范围内，由一个机械过程产生；这个过程取决于讲话人的声道、舌头和下颚。一些声码器利用发声系统的模型来减少语音，只需要使用几个参数（例如，各种腔的大小和形状）和低至 2.4 kbps 的数据传输速率。然而，这些声码器是如何工作的超出了本书的范围，这里不再赘述。

我们主要关注通过 Internet 发送音频，一种通常接近 CD 音质的音频。通过 Internet 发送时必须减少这种音频的数据传输速率。1.411 Mbps 的立体声音频会占用许多宽带链路，给视频和其他 Web 流量留下的余地很少。音频压缩后的数据率可降低一个数量级，而用户几乎感觉不到音质的损失。

压缩和解压需要对信号进行处理。幸运的是，数字化的声音和电影可以很容易由计算机的软件处理。事实上，完成这件事情的程序已经有数十个，它们允许用户记录、显示、编辑、合成和存储来自多个源的流媒体。这种技术的应用已经导致了大量的音乐和电影可以通过 Internet 唾手可得——当然，并不是所有这些都是合法的——因此，这又导致了許多来自艺术家和版权拥有人的诉讼。

已经开发了许多音频压缩算法。可能最流行的格式是 **MPEG 音频层 3** (MP3, MPEG audio layer 3) 和高级音频编码 (AAC, Advanced Audio Coding)，后者是由 MP4 (MPEG-4) 文件给出的。为了避免混淆，需要注意的是 MPEG 提供了音频和视频压缩。MP3 是指 MPEG-1 标准中的音频压缩部分 (第三部分)，而不是第三个版本的 MPEG。事实上，没有发布过 MPEG 的第三个版本，只有 MPEG-1、MPEG-2 和 MPEG-4。AAC 是 MP3 的继任者；也是 MPEG-4 中使用的默认音频编码。MPEG-2 允许 MP3 和 AAC 音频。现在说清楚了吗？有关标准的好处就是有这么多的选择。如果你不喜欢它们之中的任何一种，只有再等上一年或两年。

音频压缩可以通过两种方式完成。在**波形编码** (waveform coding) 中，它通过傅里叶变换，将信号数学地转换成频率分量。在第 2 章中，我们用图 2-1(a)所示的例子显示了一个时间函数以及它的傅里叶变换。然后以最小的方式对每个分量的强度进行编码。这样做的目标是尽可能在另一端以尽可能少的位数重现该波形。

另一种方式称为**感知编码** (perceptual coding)，它利用了人类听觉系统中的某些特定缺陷来对信号进行编码，即使通过示波器观察到的信号与原信号有相当大的差异，但在人耳听起来还是一样的。感知编码方法建立在**心理声学** (psychoacoustics) 的基础上，即人类是如何感觉到声音的。MP3 和 AAC 都是基于感知编码的。

感知编码的关键特性是某些声音可以**屏蔽** (mask) 其他声音。请想象在一个温暖的夏日里，你正在现场直播一场长笛音乐会。然后，突然间，附近的一群工人打开他们的手提电钻开始在街道上施工。于是，没有人能够听得到长笛的声音。长笛的声音被手提电钻完全屏蔽了。如果仅仅考虑传输的话，则只需对手提电钻所使用的频段进行编码就足够了，因为听众反正也听不到长笛的声音。这种情形称为**频率屏蔽** (frequency masking)——某一频段中较大的声音隐藏了另一个频段中较柔和的声音，这种能力即为频率屏蔽。事实上，即使手提电钻停下来，也有很短的一段时间听不到长笛的声音，因为当手提电钻工作的时候耳朵降低了它的增益，它需要一段有限的时间再次提高增益。这种效果称为**暂时屏蔽** (temporal masking)。

要使这些效果更明显，请想象做实验 1。一个人在一个安静的房间里，戴着一个连接到计算机声卡的耳机。计算机生成一个较弱的 100 Hz 的纯正弦波信号，然后逐渐增加功率。下达的科目要求他听到声音的时候就敲击一个键。计算机记录下此时的功率值，然后用 200 Hz、300 Hz 和其他所有人类听力范围以内的频率重复这个实验。把多个人的数据进行平均，从而得到了如图 7-43(a)所示的一个对数 (log-log) 图，它表示一个音调被人耳听见

需要多少功率。根据图中的曲线,可以直接得出的一个结论是永远不需要对一个功率低于可听度阈值的频率进行编码。例如,在图 7-43(a)中。如果 100 Hz 的功率是 20 dB,那么可以在输出中省略掉它而不会产生可感觉的品质损失,因为在 100 Hz 频率上 20 dB 低于可听度水平。

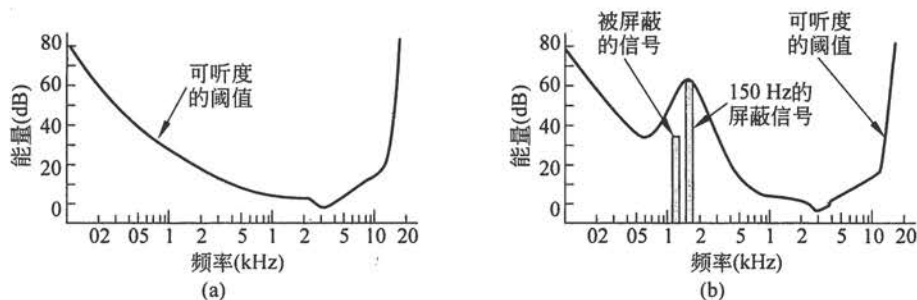


图 7-43

(a) 作为频率函数的可听度阈值; (b) 屏蔽的效果

现在考虑实验 2。计算机再次运行实验 1,但这次在测试频率上叠加了一个固定振幅的正弦波,它的频率是 150 Hz。我们发现,在 150 Hz 附近频率的可听度阈值增加了,如图 7-43(b)所示。

这次新观测的结果是记录下哪些信号被附近频段中更强的信号所屏蔽,然后在编码信号时省略掉这些越来越多的频率,从而节省数据位。在图 7-43 中,125 Hz 的信号可以在输出中被完全省略,而且没有人能够听得出差异来。在某个频段中,即使当一个功率较强的信号停止以后,考虑到它的暂时屏蔽特性,我们可以继续省略被屏蔽掉的频率一段时间(即耳朵恢复听力的时间)。MP3 和 AAC 的本质就是对声音进行傅里叶变换以便得到每个频率处的功率,然后只传输未被屏蔽的频率,并且用尽可能少的位数对它们进行编码。

有了这个信息作为背景,我们现在可以来看编码是如何完成的。音频压缩是以 8~96 的采样率对波形进行采样,以此模仿 CD 音质的声音,一般的采样率为 44.1 kHz。采样可以对一个信道(单声道)或两个(立体声)信道进行。下一步要选择输出比特率。MP3 可以将一个立体声摇滚乐光盘压缩到 96 kbps,带来感官上的音质损失很小,甚至摇滚乐迷也听不大出来。对于一场钢琴音乐会,至少需要 128 kbps 的 AAC。这两种情况下需要的比特率之所以不同的原因在于摇滚乐的信号信噪比要比钢琴演奏会的信噪比高得多(无论如何这是工程上的意义)。它也可以选择较低的输出率,并且接受某些质量上的损失。

采样值以小批量的形式处理。每个批次的信号通过一排数字滤波器以便获得频段。接着频率信息再被送入一个心理声学模型,目的是确定屏蔽掉的频率;然后,对可用比特在频段之间进行划分:给具有最大频谱功率的无屏蔽频段分配更多的比特,给具有小频谱功率的无屏蔽频段分配较少的比特,不给任何被屏蔽的频段分配任何比特。最后,使用 Huffman 编码算法对比特进行编码,给经常出现的数字分配短码,给那些不经常发生的数字分配长码。对此具有好奇心的读者可能希望介绍更多的细节。欲了解有关的更多信息,请参阅(Brandenburg, 1999)。

7.4.2 数字视频

现在我们知道了所有关于耳朵的事情，是时间把注意力转移到眼球上了（当然本节不会在眼睛之后再转到鼻子上）。人类的眼睛有个特点，当一幅图像出现在视网膜上时，图像在退化之前会在视网膜上停留数毫秒。如果一个图像序列以每秒 50 幅的速率出现，眼睛是不会注意到这些图像是离散的。所有视频系统正是利用了这个原理来产生动画效果的。

视频最简单的数字表示法是一个帧序列，每个帧包含一个由图像元素或者像素（pixel）组成的矩形网格。每个像素可以是一个单独的位，表示黑或者白。然而，这样一个系统的质量极差。你可以试着用一个熟悉的图像编辑器将一张彩色图像的相片转换成黑白形式（而且不能有灰度阴影），看看效果如何。

接下来，每个像素用 8 位来表示 256 个灰度级。这种方案可以表示高质量的黑白（black-and-white）视频。对于彩色视频，许多系统使用 8 位来表示红、绿和蓝（RGB）三原色中的每一个分量。这种表示法是可能的，因为任何一种颜色可以通过线性叠加适当强度的红、绿、蓝构造出来。每个像素用 24 位来表示，就有大概 1600 万种颜色，远远超出了人类眼睛可以识别的范围。

在彩色 LCD 计算机显示器和电视机上，每个离散的像素是由紧密排列的红色、绿色和蓝色子像素组成的。帧由子像素的强度来显示，而眼睛混合了颜色分量。

常见的帧速率是 24 帧/秒（继承自 35 毫米的电影胶片）、30 帧/秒（取自 NTSC 美国电视）和 25 帧/秒（取自几乎全世界都在用的 PAL 电视系统）。（对于真正的挑剔者来说，NTSC 彩色电视的确切运行速率是 29.97 帧/秒。最原始的黑白系统以 30 帧/秒速率运行，但在引入颜色时，工程师需要一点额外的带宽作为信令，所以他们将帧速率减少到 29.97。NTSC 制式的视频在计算机上真正使用的帧速率还是 30）。PAL 制式在 NTSC 之后被发明出来，实际使用 25.000 帧/秒。为了使得这个故事看起来更完整，还有第三个系统，即法国、法属非洲和东欧使用的 SECAM。它是由当时的共产主义东德引进到东欧的，因此东德人无法观看西德（PAL）电视，以免他们得到不好的想法。但是现在许多东欧国家正在往 PAL 制式切换。技术和政治两个方面都在尽力而为地努力着。

事实上，对于广播电视而言，用 25 帧/秒的速率来播放流畅的动作效果还不够好，因此图像被分割分为两个域（field），一个由奇数扫描线构成，另一个由偶数扫描线构成。两个域（一半的分辨率）按顺序播放，给出接近 60（NTSC）/秒或确切地 50（PAL）域/秒的速率，这样的系统就称为隔行扫描（interlacing）。在计算机上观看的视频是渐进的（progressive），也就是说不使用隔行扫描，因为计算机显示器在图形卡上有个缓冲区，可使得 CPU 以 30 次/秒的速率将新图像放入缓冲区中，但图形卡以 50 或者甚至 100 次/秒的速率重绘屏幕来消除闪烁。模拟电视机没有像计算机那样的帧缓冲区。当快速运动的隔行扫描视频显示在计算机上时，在阴影边缘能隐约看到短的水平线，这就是称为梳理（combing）的效果。

通过 Internet 发送视频的帧的大小差异很大，个中的原因很简单，因为较大的帧需要更多的带宽，而这个要求可能并不总能满足。低分辨率的视频可能只有 320×240 像素，“全屏”视频是 640×480 像素。这些尺寸分别近似于早期的计算机显示器和 NTSC 电视。宽高

比 (aspect ratio), 或宽度与高度的比例为 4:3, 与标准电视一样。高清晰电视 (HDTV, High-Definition TeleVision) 视频可下载为 1280×720 像素。这些“宽屏”图像的宽高比为 16:9, 更加接近电影的 3:2 宽高比。相比之下, 标准的 DVD 视频通常是 720×480, 蓝光光盘的视频通常是高清 1080×720。

在 Internet 上, 像素的数目仅是故事的一部分, 因为媒体播放器可以不同的大小来表示相同的图像。视频只是在计算机屏幕上的另一个窗口, 它可以被拉大或缩小。像素越多, 其作用是提高了图像质量, 所以当它被扩大时看起来并不模糊。然而, 许多显示器可以显示甚至比 HDTV 更多的像素图像 (和视频)。

视频压缩

从我们关于数字视频的讨论可以很明显地看出, 通过 Internet 发送视频的关键是压缩。即使是标准质量的视频, 即帧具有 640×480 像素、每个像素有 24 位的颜色信息、播放速率为 30 帧/秒, 这样的视频也需要超过 200 Mbps。这个数字远远超过了大多数公司办公室连接到 Internet 的带宽, 更不用说家庭用户; 而且这还只是单一视频流。由于传输无压缩视频几乎根本不可能, 至少在广域网络上如此, 那么唯一的希望是进行大规模的压缩。幸运的是, 过去几十年大量的研究已经产生了许多压缩技术和算法, 它们使得通过 Internet 传输视频成为可能。

通过 Internet 发送的视频使用了多种格式, 一些是专有的, 还有一些是标准化的。最流行的编码是各种形式的 MPEG。它是一个开放标准, 适用以 mpg 和 mp4 为扩展名的文件, 以及其他容器格式的文件。在本节, 我们将通过 MPEG 来学习如何完成视频压缩。首先, 我们将着眼于用 JPEG 对静止图像进行压缩。一个视频仅仅是一个图像序列 (加上声音)。压缩视频的方式之一就是连续编码每一幅图像。第一个近似方法就是 MPEG, 它只是对每一帧进行 JPEG 编码, 再加上一些额外的功能, 比如消除跨帧的冗余度。

JPEG 标准

联合图像专家组 (JPEG, Joint Photographic Experts Group) 标准用来压缩连续色调的静止图像 (比如照片), 该标准是由图像专家组在 ITU、ISO 和 IEC (另一个标准化组织) 的共同主持下完成的。它已得到广泛应用 (扩展的 jpg 文件), 并且经常提供的压缩比为 10:1, 对自然图像的压缩效果更好。

JPEG 由国际标准 10918 定义。实际上, 相比一个单一的算法, 它更像一个购物清单。它所定义四个模式中, 只有一种有损顺序模式与我们的讨论有关。我们将重点关注 JPEG 被通常用于 24 位 RGB 视频图像编码的方式, 而且为简单起见还会留下一些选项和细节供读者自己进一步查阅。

该算法如图 7-44 所示。第 1 步是块准备。为了说明特征, 让我们假设 JPEG 的输入是一幅 640×480 的 RGB 图像, 具有 24 位/像素, 如图 7-45 (a) 所示。RGB 并不是压缩使用的最好颜色模型。视频信号中的眼睛对亮度 (luminance) 或亮光, 比对色度 (chrominance) 或颜色要敏感得多。因此, 我们首先从 R、G 和 B 分量计算亮度 Y, 然后再计算两个色度 Cb 和 Cr。下面公式用于 8 位值, 变化范围从 0 到 255:

$$Y = 16 + 0.26R + 0.50G + 0.09B$$

$$Cb = 128 + 0.15R - 0.29G - 0.44B$$